Claude Implementation

AI Learning Lab - Implementation Tasks

Project: AI Learning Lab - Hyper-personal daily learning system

PRD Reference: `ai-learning-lab-prd-cc.md`

Target: Complete product implementation (All Phases)

Tool: Claude Code CLI

Branch Strategy: Single feature branch for entire project

Instructions for Completing Tasks

IMPORTANT: As you complete each task, you must check it off in this markdown file by changing `- [ ]` to `- [x]`. This helps track progress and ensures you don't skip any steps.

Example:

- `- [ ] 1.1 Read file` → `- [x] 1.1 Read file` (after completing)

Update the file after completing each sub-task, not just after completing an entire parent task.

How to Use This Task List with Claude Code

Getting Started

1. **Install Claude Code CLI** (if not already installed):

```bash
npm install -g @anthropic-ai/claude-code
```

2. **Start Claude Code** in your project directory:

```bash
cd /path/to/your/project
claude-code
```

3. **For each task:**

- Read the task description and "What we're doing" section

- Copy the provided prompt and paste it into Claude Code CLI

- Review the output and verify using the "Expected outcome" and "Verification" steps

- Check off the task by editing this file: `- [ ]` → `- [x]`

Understanding the Task Format

Each sub-task includes:

- ****What we're doing:**** Context and purpose
- ****Tool:**** Which Claude Code feature to use (CLI, web, etc.)
- ****Prompt to Claude:**** Exact text to paste (copy-paste ready)
- ****Expected outcome:**** What should happen when successful
- ****Example output:**** Sample of what you'll see
- ****Verification:**** How to confirm it worked

Tasks

0.0 Create Feature Branch

****What we're doing:**** Setting up version control for the entire AI Learning Lab implementation. We'll create a dedicated feature branch to keep all development work isolated from the main branch until the product is complete, tested, and ready for deployment.

- [ ] ****0.1 Initialize git repository (if not already done)****

  ****What we're doing:**** Checking if git is already initialized in your project directory. If not, we'll create a new repository.

  ****Tool:**** Claude Code CLI

  ****Prompt to Claude:****

  ```
  Check if this directory is already a git repository by running: git status

  If it says "not a git repository", initialize git:

  git init

  Then check the status again and show me the output.
  ```

  ****Expected outcome:**** Git repository initialized (if needed)

  ****Example output:****

  ```
  Initialized empty Git repository in /path/to/ai-learning-lab/.git/

  On branch main

  No commits yet
  ```

  ****Verification:**** Run `git status` - should not show "not a git repository" error

- [ ] ****0.2 Create and checkout feature branch****

  ****What we're doing:**** Creating a new branch called `feature/ai-learning-lab` where all our development will happen.

Tool: Claude Code CLI

Prompt to Claude:

```
Create and checkout a new feature branch for the AI Learning Lab:

git checkout -b feature/ai-learning-lab

Then show the current branch:

git branch

Show me the output.
```

Expected outcome: New branch created and checked out

Example output:

```
Switched to a new branch 'feature/ai-learning-lab'

* feature/ai-learning-lab
```

Verification: `git branch` shows `* feature/ai-learning-lab` (asterisk indicates current branch)

1.0 Project Setup & Infrastructure

What we're doing: Building the foundation for our AI Learning Lab application. We'll initialize a Next.js 14 project with TypeScript, install all necessary dependencies (Shadcn/ui for beautiful components, Prisma for database access, NextAuth for authentication, Groq SDK for AI features, and Framer Motion for animations), create an organized directory structure, set up Docker for local PostgreSQL database, and configure code quality tools. This foundation ensures we have a solid, professional development environment before writing any feature code.

- [ ] **1.1 Initialize Next.js 14 project with TypeScript**

What we're doing: Creating a new Next.js 14 application using the App Router (the modern Next.js architecture) with TypeScript for type safety and Tailwind CSS for styling.

Tool: Claude Code CLI

Prompt to Claude:

```
Create a new Next.js 14 project with these specifications:

Run: npx create-next-app@latest ai-learning-lab

When prompted, answer:

- Would you like to use TypeScript? → Yes

- Would you like to use ESLint? → Yes

- Would you like to use Tailwind CSS? → Yes
```

- Would you like to use `src/` directory? → No (we'll add it manually later for better organization)

- Would you like to use App Router? → Yes

- Would you like to customize the default import alias (@/*)? → No

After creation, cd into the project and show me the file structure:

cd ai-learning-lab

ls -la

```

**Expected outcome:** Next.js project created with TypeScript, Tailwind, and App Router

**Example output:**

```

Creating a new Next.js app in /path/to/ai-learning-lab...

✔ Would you like to use TypeScript? … Yes

✔ Would you like to use ESLint? … Yes

✔ Would you like to use Tailwind CSS? … Yes

✔ Would you like to use `src/` directory? … No

✔ Would you like to use App Router? … Yes

✔ Would you like to customize the default import alias? … No

Success! Created ai-learning-lab at /path/to/ai-learning-lab

total 48

drwxr-xr-x app/

-rw-r--r-- next.config.mjs

-rw-r--r-- package.json

-rw-r--r-- tailwind.config.ts

-rw-r--r-- tsconfig.json

```

**Verification:** Check for `app/` directory, `tailwind.config.ts`, `tsconfig.json` files

- [ ] **1.2 Install required dependencies**

**What we're doing:** Installing all the npm packages our application needs - Prisma for database, NextAuth for authentication, Groq SDK for AI, Zod for validation, and Framer Motion for animations.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Install all required dependencies for the AI Learning Lab:

Core dependencies:

npm install @prisma/client next-auth@beta groq-sdk zod framer-motion

Dev dependencies:

npm install -D prisma @types/node @types/react @types/react-dom

After installation, show me the dependencies section from package.json:

cat package.json | grep -A 20 "dependencies"

```

**Expected outcome:** All packages installed successfully

**Example output:**

```

added 127 packages, and audited 340 packages in 12s

"dependencies": {

  "@prisma/client": "^5.7.0",

  "framer-motion": "^10.16.16",

  "groq-sdk": "^0.3.0",

  "next": "14.0.4",

  "next-auth": "5.0.0-beta.4",

  "react": "^18.2.0",

  "react-dom": "^18.2.0",

  "zod": "^3.22.4"

},

"devDependencies": {

  "@types/node": "^20",

  "@types/react": "^18",

  "@types/react-dom": "^18",

  "prisma": "^5.7.0"

}

```

**Verification:** All packages listed in package.json dependencies

- [ ] **1.3 Setup Shadcn/ui component library**

**What we're doing:** Installing Shadcn/ui, a collection of beautifully designed, accessible UI components built on Radix UI. This will give us professional-looking buttons, cards, forms, and modals that match our minimalist design aesthetic.

**Tool:** Claude Code CLI

**Prompt to Claude:**
```

```

Setup Shadcn/ui for this Next.js project:

1. First, run the init command:

npx shadcn-ui@latest init

When prompted, select:

- Which style would you like to use? → Default

- Which color would you like to use as base color? → Slate

- Would you like to use CSS variables for colors? → Yes

2. After setup, install these essential components we'll need:

npx shadcn-ui@latest add button card input label checkbox radio-group select textarea dialog skeleton

3. Show me the created components directory:

ls -R components/

```

****Expected outcome:**** Shadcn/ui configured with components installed

****Example output:****

```

✔ Setting up dependencies...

✔ Writing components.json...

✔ Initializing project...

✔ Installing components...

Success! Components installed at components/ui/

components/:

ui/

components/ui/:

button.tsx

card.tsx

checkbox.tsx

dialog.tsx

input.tsx

label.tsx

radio-group.tsx

select.tsx

skeleton.tsx

textarea.tsx
```

```

**Verification:** Check that `components/ui/` directory exists with all component files

- [ ] **1.4 Create src/ directory structure**

**What we're doing:** Organizing our codebase with a clear directory structure. We'll move the `app/` directory into a new `src/` folder and create subdirectories for components, utilities, types, and more. This structure (from the PRD) helps us keep the code organized as the project grows.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the recommended file structure from the PRD:

1. Create the src/ directory and move app/ into it:

mkdir src

mv app src/

mv components src/

2. Create the full directory structure:

mkdir -p src/components/onboarding

mkdir -p src/components/learning

mkdir -p src/components/memory

mkdir -p src/components/shared

mkdir -p src/lib/db

mkdir -p src/lib/groq

mkdir -p src/lib/auth

mkdir -p src/lib/utils

mkdir -p src/types

mkdir -p src/styles

3. Update tsconfig.json to include the src directory in paths.

   Add or update the "baseUrl" to "src" and ensure paths are correct.

4. Show me the complete structure:

tree src/ -L 2 -d

If tree is not installed, use:

find src -type d -maxdepth 2

```

**Expected outcome:** Organized src/ directory structure

**Example output:**
```

```

src/

├── app

├── components

│ ├── ui

│ ├── onboarding

│ ├── learning

│ ├── memory

│ └── shared

├── lib

│ ├── db

│ ├── groq

│ ├── auth

│ └── utils

├── types

└── styles

```

Verification: Run `ls src/` - should see app, components, lib, types, styles

- [ ] **1.5 Setup Prisma with PostgreSQL**

What we're doing: Initializing Prisma, our database ORM (Object-Relational Mapping) tool. Prisma will help us interact with PostgreSQL using TypeScript instead of writing raw SQL queries. We'll create the initial schema file and configure the database connection.

Tool: Claude Code CLI

Prompt to Claude:

```

Initialize Prisma for PostgreSQL:

1. Run Prisma init:

npx prisma init --datasource-provider postgresql

This creates:

- prisma/schema.prisma (database schema definition)

- .env (environment variables)

2. Update the .env file with local PostgreSQL connection string.

Replace the existing DATABASE_URL with:

DATABASE_URL="postgresql://postgres:postgres@localhost:5432/ai_learning_lab?schema=public"
```

3. Create the Prisma client singleton at src/lib/db/prisma.ts to avoid

multiple instances in development:

Create a file with this content (handles hot reloading properly):

import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {

prisma: PrismaClient | undefined

}

export const prisma =

globalForPrisma.prisma ??

new PrismaClient({

log: ['query'],

})

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma

4. Show me the created files:

ls -la prisma/

cat .env

cat src/lib/db/prisma.ts

```

**Expected outcome:** Prisma initialized with PostgreSQL configuration

**Example output:**

```

✔ Your Prisma schema was created at prisma/schema.prisma

✔ Environment variables loaded from .env

prisma/:

schema.prisma

.env file:

DATABASE_URL="postgresql://postgres:postgres@localhost:5432/ai_learning_lab?schema=public"

src/lib/db/prisma.ts created successfully

```

**Verification:** Check for `prisma/schema.prisma`, `.env`, and `src/lib/db/prisma.ts`

- [ ] **1.6 Create Docker Compose for local PostgreSQL**

**What we're doing:** Setting up Docker Compose to run PostgreSQL database locally. This way, you don't need to install PostgreSQL on your machine - Docker will handle it. We'll also create a `.env.example` file as a template for other developers.
```

Tool: Claude Code CLI

Prompt to Claude:

```
Create Docker configuration for local PostgreSQL:

1. Create docker-compose.yml in the project root with this content:

version: '3.8'

services:

  postgres:

    image: postgres:15-alpine

    container_name: ai-learning-lab-db

    restart: always

    environment:

      POSTGRES_USER: postgres

      POSTGRES_PASSWORD: postgres

      POSTGRES_DB: ai_learning_lab

    ports:

      - "5432:5432"

    volumes:

      - postgres_data:/var/lib/postgresql/data

volumes:

  postgres_data:

2. Create .env.example with all required environment variables:

# App

NEXT_PUBLIC_APP_URL=http://localhost:3000

NODE_ENV=development

# Database

DATABASE_URL="postgresql://postgres:postgres@localhost:5432/ai_learning_lab?schema=public"

# Auth (NextAuth.js)

NEXTAUTH_SECRET=your-secret-key-here-generate-with-openssl-rand-base64-32

NEXTAUTH_URL=http://localhost:3000

# OAuth Providers (Get from Google Cloud Console and GitHub Settings)

GOOGLE_CLIENT_ID=your-google-client-id

GOOGLE_CLIENT_SECRET=your-google-client-secret

GITHUB_CLIENT_ID=your-github-client-id
```

GITHUB_CLIENT_SECRET=your-github-client-secret

Groq API (Get from https://console.groq.com)

GROQ_API_KEY=your-groq-api-key

GROQ_MODEL_REASONING=llama-3.1-70b-versatile

GROQ_MODEL_FAST=mixtral-8x7b-instant-v0.1

3. Show me both files:

cat docker-compose.yml

cat .env.example

```

**Expected outcome:** Docker Compose and environment template created

**Example output:**

```

docker-compose.yml created successfully

.env.example created successfully

version: '3.8'

services:

  postgres:

    image: postgres:15-alpine

    ...

```

**Verification:** Files `docker-compose.yml` and `.env.example` exist in project root

- [ ] **1.7 Start PostgreSQL container and verify connection**

**What we're doing:** Launching the PostgreSQL database using Docker and verifying that Prisma can connect to it successfully. This confirms our database is ready for development.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Start PostgreSQL and verify the connection:

1. Make sure Docker Desktop is running, then start the container:

docker-compose up -d

2. Wait 5 seconds for the database to be ready, then check if it's running:

sleep 5

docker ps

3. Test Prisma connection (this will sync the empty schema):

npx prisma db push

4. Show me the output of each command.

```

**Expected outcome:** PostgreSQL container running, Prisma connected successfully

**Example output:**

```

[+] Running 2/2

✔ Network ai-learning-lab_default Created

✔ Container ai-learning-lab-db Started

CONTAINER ID IMAGE COMMAND STATUS

abc123def456 postgres:15-alpine "docker-entrypoint.s…" Up 5 seconds

Prisma schema loaded from prisma/schema.prisma

Datasource "db": PostgreSQL database "ai_learning_lab"

✔ Generated Prisma Client

```

**Verification:** `docker ps` shows `ai-learning-lab-db` container running

- [ ] **1.8 Configure ESLint and Prettier**

**What we're doing:** Setting up code formatting and linting tools. Prettier will automatically format our code consistently, and ESLint will catch common errors. This ensures our codebase stays clean and professional.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Setup code quality tools:

1. Install Prettier and integrate with ESLint:

npm install -D prettier eslint-config-prettier

2. Create .prettierrc.json with these settings:

{

"semi": true,

"singleQuote": false,

"tabWidth": 2,

"trailingComma": "es5",

"printWidth": 100,

"arrowParens": "always"

}

3. Update .eslintrc.json to extend prettier (add "prettier" to extends array)

4. Add format scripts to package.json scripts section:

"format": "prettier --write "*/.{ts,tsx,js,jsx,json,md}"",

"format:check": "prettier --check "*/.{ts,tsx,js,jsx,json,md}"",

"lint": "next lint"

5. Run the formatter on all files:

npm run format

6. Show me the results.

```

**Expected outcome:** Prettier configured, all files formatted

**Example output:**

```

added 12 packages in 3s

.prettierrc.json created

.eslintrc.json updated

package.json updated with format scripts

Formatting...

src/app/layout.tsx 150ms

src/app/page.tsx 45ms

tailwind.config.ts 32ms

✔ 27 files formatted

```

**Verification:** Run `npm run format:check` - should show "All matched files use Prettier code style!"

- [ ] **1.9 Create initial git commit**

**What we're doing:** Saving all the foundation work we've done so far in a git commit. This creates a checkpoint we can return to if needed.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the initial commit with all setup files:

1. First, create a .gitignore file if it doesn't exist or update it to include:

node_modules/

.env

.next/

dist/

.vercel

postgres_data/

2. Check what files will be committed:

git status

3. Stage all files:

git add .

4. Create the commit:

git commit -m "chore: initial project setup

- Initialize Next.js 14 with App Router and TypeScript

- Setup Tailwind CSS and Shadcn/ui component library

- Configure Prisma ORM with PostgreSQL

- Add Docker Compose for local database

- Setup ESLint and Prettier for code quality

- Create recommended directory structure from PRD

🤖 Generated with Claude Code"

5. Show the commit log:

git log -1 --stat

```

**Expected outcome:** Initial commit created successfully

**Example output:**

```

[feature/ai-learning-lab abc1234] chore: initial project setup

 42 files changed, 3847 insertions(+)

 create mode 100644 docker-compose.yml

 create mode 100644 package.json

 create mode 100644 prisma/schema.prisma

 ...

commit abc1234def5678

Author: Your Name you@example.com

Date: Wed Dec 25 2025

    chore: initial project setup

```

**Verification:** `git log` shows the commit with all files
```

2.0 Database Schema & Authentication System

****What we're doing:**** Creating the data foundation for our application. We'll define all 11 database models (tables) in Prisma schema - everything from User accounts to Learning topics to Memory entries. Then we'll set up NextAuth.js v5, a powerful authentication library, and configure it to work with both Google and GitHub OAuth (so users can sign in with their existing accounts). This is critical infrastructure that every other feature will depend on.

- [ ] ****2.1 Create complete Prisma schema with all models****

****What we're doing:**** Defining the entire database structure with all 11 models from the PRD. This is the single source of truth for our data model - Prisma will use this to create database tables and generate TypeScript types.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Replace the contents of prisma/schema.prisma with the complete schema from the PRD (section 8):

Create the schema with these 11 models:

1. User (id, email, name, timestamps)

2. AuthAccount (OAuth provider accounts)

3. UserProfile (all 27 questionnaire answers)

4. LearningStrategy (derived course design decisions)

5. Topic (user's learning topics)

6. TopicGraphVersion (immutable concept graphs)

7. Concept (individual learning nodes)

8. DailyPlan (sequenced learning schedule)

9. MemoryEntry (learning records)

10. EvidenceItem (proof-of-work attachments)

11. GitHubConnection (GitHub OAuth for evidence)

Include all fields, relations, and indexes as specified in PRD section 8.

After creating the schema, show me the file:

cat prisma/schema.prisma

```

****Expected outcome:**** Complete Prisma schema with all 11 models

****Example output:****

```prisma

generator client {

  provider = "prisma-client-js"

}
```

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
model User {
  id            String          @id @default(cuid())
  email         String          @unique
  name          String?
  createdAt     DateTime        @default(now())
  updatedAt     DateTime        @updatedAt
  authAccounts  AuthAccount[]
  profile       UserProfile?
  topics        Topic[]
  memoryEntries MemoryEntry[]
  githubConn    GitHubConnection?
}
model AuthAccount {
  id             String   @id @default(cuid())
  userId         String
  provider       String
  providerUserId  String
  accessToken     String?  @db.Text
  refreshToken    String?  @db.Text
  expiresAt       DateTime?
  user           User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([provider, providerUserId])
  @@index([userId])
}
... (continue with all 11 models)
```

****Verification:**** Schema file contains all 11 models with proper relations

- [ ] ****2.2 Generate Prisma Client****

****What we're doing:**** Running Prisma's code generator to create TypeScript types and database client based on our schema. This gives us type-safe database access throughout the application.

Tool: Claude Code CLI

Prompt to Claude:

```
Generate the Prisma Client from our schema:

npx prisma generate

This will:

- Read prisma/schema.prisma

- Generate TypeScript types for all models

- Create the Prisma Client in node_modules/@prisma/client

Show me the output and confirm that types were generated.
```

Expected outcome: Prisma Client generated successfully with TypeScript types

Example output:

```
Prisma schema loaded from prisma/schema.prisma

✔ Generated Prisma Client (5.7.0) to ./node_modules/@prisma/client

You can now import the Prisma Client:

import { PrismaClient } from '@prisma/client'
```

Verification: No errors in output, can import PrismaClient in code

- [ ] **2.3 Run first migration**

What we're doing: Creating the database tables based on our Prisma schema. This migration will create all 11 tables in PostgreSQL with the correct columns, indexes, and relationships.

Tool: Claude Code CLI

Prompt to Claude:

```
Create and run the first database migration:

npx prisma migrate dev --name init

This will:

1. Create a migration file in prisma/migrations/

2. Apply the migration to create all tables in PostgreSQL

3. Regenerate Prisma Client

Show me the output and the created migration folder:

ls prisma/migrations/
```

```

**Expected outcome:** Migration created and applied, all tables exist in database

**Example output:**

```

Prisma Migrate created and applied the following migration:

migrations/

└─ 20251225000000_init/

└─ migration.sql

✔ Generated Prisma Client

Database synchronized with Prisma schema.

```

**Verification:** `prisma/migrations/` folder exists with migration file

- [ ] **2.4 Create Prisma client utility with proper singleton**

**What we're doing:** Updating our Prisma client utility to handle Next.js hot reloading properly. This prevents creating multiple database connections during development.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Verify and update src/lib/db/prisma.ts with the singleton pattern:

The file should contain:

import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {

prisma: PrismaClient | undefined

}

export const prisma =

globalForPrisma.prisma ??

new PrismaClient({

log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error'],

})

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma

Show me the file and confirm it matches this pattern.

```

**Expected outcome:** Singleton pattern implemented correctly

**Example output:**
```

```

src/lib/db/prisma.ts:

import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as {

  prisma: PrismaClient | undefined

}

export const prisma = ...

```

Verification: File exists at `src/lib/db/prisma.ts` with singleton pattern

- [ ] **2.5 Install and configure NextAuth.js v5**

What we're doing: Setting up NextAuth.js version 5 (beta), the authentication library we'll use for OAuth login. We'll create the core configuration file that defines how authentication works in our app.

Tool: Claude Code CLI

Prompt to Claude:

```

Setup NextAuth.js v5 configuration:

1. Verify next-auth beta is installed (we did this in 1.2):

npm list next-auth

2. Create the NextAuth configuration at src/lib/auth/auth.config.ts:

import type { NextAuthConfig } from "next-auth"

import Google from "next-auth/providers/google"

import GitHub from "next-auth/providers/github"

export const authConfig: NextAuthConfig = {

  providers: [

    Google({

      clientId: process.env.GOOGLE_CLIENT_ID!,

      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,

    }),

    GitHub({

      clientId: process.env.GITHUB_CLIENT_ID!,

      clientSecret: process.env.GITHUB_CLIENT_SECRET!,

    }),

  ],

  pages: {
```

```
    signIn: "/login",
  },
  callbacks: {
    authorized({ auth, request: { nextUrl } }) {
      const isLoggedIn = !!auth?.user
      const isOnDashboard = nextUrl.pathname.startsWith("/dashboard")
      const isOnOnboarding = nextUrl.pathname.startsWith("/onboarding")
      if (isOnDashboard || isOnOnboarding) {
        if (isLoggedIn) return true
        return false // Redirect to login
      }
      return true
    },
  },
}
```

3. Create src/lib/auth/auth.ts for the main auth instance:

```
import NextAuth from "next-auth"
import { PrismaAdapter } from "@auth/prisma-adapter"
import { prisma } from "@/lib/db/prisma"
import { authConfig } from "./auth.config"
export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(prisma),
  session: { strategy: "jwt" },
  ...authConfig,
})
```

4. Install the Prisma adapter:

```
npm install @auth/prisma-adapter
```

5. Show me both created files.

```

**Expected outcome:** NextAuth.js configured with Google and GitHub providers

**Example output:**

```
next-auth@5.0.0-beta.4
src/lib/auth/auth.config.ts created
```
```

src/lib/auth/auth.ts created

@auth/prisma-adapter@1.0.12 installed

```

**Verification:** Files exist at `src/lib/auth/auth.config.ts` and `src/lib/auth/auth.ts`

- [ ] **2.6 Create NextAuth API route handler**

**What we're doing:** Creating the Next.js API route that NextAuth uses to handle authentication requests (login, logout, callbacks). This is required for NextAuth to function.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the NextAuth.js API route:

1. Create the route handler at src/app/api/auth/[...nextauth]/route.ts:

import { handlers } from "@/lib/auth/auth"

export const { GET, POST } = handlers

This exports the NextAuth request handlers for both GET and POST requests.

2. Show me the file and directory structure:

cat src/app/api/auth/[...nextauth]/route.ts

tree src/app/api/ -L 2

```

**Expected outcome:** NextAuth API route created

**Example output:**

```

src/app/api/auth/[...nextauth]/route.ts:

import { handlers } from "@/lib/auth/auth"

export const { GET, POST } = handlers

src/app/api/

└── auth/

    └── [...nextauth]/

        └── route.ts

```

**Verification:** File exists at `src/app/api/auth/[...nextauth]/route.ts`

- [ ] **2.7 Update Prisma schema for NextAuth adapter**

**What we're doing:** NextAuth with Prisma adapter expects certain models in our database. We need to ensure our User and AuthAccount models match NextAuth's requirements.
```

Tool: Claude Code CLI

Prompt to Claude:

```
Verify our Prisma schema is compatible with NextAuth Prisma adapter:
The User and AuthAccount models should have these fields at minimum:
User:
- id (String, @id)
- email (String, @unique)
- emailVerified (DateTime?, optional)
- name (String?, optional)
- image (String?, optional)
Session model (for database sessions, though we're using JWT):
- Not needed if using JWT strategy (which we are)
Account model (maps to our AuthAccount):
- userId, provider, providerAccountId, type, access_token, etc.
Check if we need to update our schema. If the models need updates, update the schema
and run: npx prisma migrate dev --name update_auth_models
Show me any changes made.
```

Expected outcome: Schema updated if needed, migration run

Example output:

```
Schema already compatible with NextAuth adapter.
User model has required fields.
AuthAccount model matches Account requirements.
No migration needed.
```

Verification: Schema compatible with NextAuth, no errors when starting app

- [ ] **2.8 Create middleware for protected routes**

What we're doing: Creating Next.js middleware that automatically protects certain routes (like dashboard, onboarding) from unauthenticated access. Users will be redirected to login if they try to access protected pages.

Tool: Claude Code CLI

Prompt to Claude:

```
```

Create middleware for route protection:

Create src/middleware.ts (must be at src/ level, not inside app/):

```
import { auth } from "@/lib/auth/auth"

export default auth((req) => {
  const { nextUrl } = req
  const isLoggedIn = !!req.auth
  const isPublicRoute =
    nextUrl.pathname === "/" ||
    nextUrl.pathname === "/login" ||
    nextUrl.pathname.startsWith("/api/auth")
  const isProtectedRoute =
    nextUrl.pathname.startsWith("/dashboard") ||
    nextUrl.pathname.startsWith("/onboarding")
  if (isProtectedRoute && !isLoggedIn) {
    return Response.redirect(new URL("/login", nextUrl))
  }
  if (isLoggedIn && nextUrl.pathname === "/login") {
    return Response.redirect(new URL("/dashboard/today", nextUrl))
  }
  return
})

export const config = {
  matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],
}
```

Show me the created file.

```

**Expected outcome:** Middleware created for route protection

**Example output:**

```

src/middleware.ts created:

```
import { auth } from "@/lib/auth/auth"

export default auth((req) => {
  ...
})
```

export const config = {

matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],

}

```

**Verification:** File exists at `src/middleware.ts`

- [ ] **2.9 Create auth utility functions**

**What we're doing:** Creating helper functions that we'll use throughout the app to check if a user is logged in, get the current user's session, etc. These utilities make auth easier to work with.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create authentication utility functions at src/lib/auth/utils.ts:

import { auth } from "./auth"

import { prisma } from "@/lib/db/prisma"

/**

 * Get the current session from server components

 */

export async function getSession() {

return await auth()

}

/**

 * Get the current user from server components

 * Throws error if not authenticated

 */

export async function getCurrentUser() {

const session = await getSession()

if (!session?.user?.email) {

throw new Error("Not authenticated")

}

const user = await prisma.user.findUnique({

where: { email: session.user.email },

include: {

profile: true,

},

```
  })
  if (!user) {
    throw new Error("User not found")
  }
  return user
}
/**
 * Check if user is authenticated
 */
export async function isAuthenticated(): Promise {
  const session = await getSession()
  return !!session?.user
}
```

Show me the created file.

```
```

Expected outcome: Auth utility functions created

Example output:

```
```

src/lib/auth/utils.ts created:

```
import { auth } from "./auth"
import { prisma } from "@/lib/db/prisma"
export async function getSession() { ... }
export async function getCurrentUser() { ... }
export async function isAuthenticated() { ... }
```

```
```

Verification: File exists at `src/lib/auth/utils.ts`

- [ ] **2.10 Commit database and auth setup**

What we're doing: Saving all our database and authentication work in a git commit.

Tool: Claude Code CLI

Prompt to Claude:

```
```

Commit the database and authentication setup:

```
git add .
```

git commit -m "feat: setup database schema and authentication

- Create complete Prisma schema with all 11 models

- Run initial database migration

- Configure NextAuth.js v5 with Google and GitHub OAuth

- Create auth utilities and protected route middleware

- Setup Prisma adapter for NextAuth

🤖 Generated with Claude Code"

git log -1 --oneline

```

**Expected outcome:** Commit created

**Example output:**

```

[feature/ai-learning-lab def5678] feat: setup database schema and authentication

 18 files changed, 847 insertions(+)

def5678 feat: setup database schema and authentication

```

**Verification:** `git log` shows new commit

---

## 3.0 Landing Page & Login Flow

**What we're doing:** Building the user-facing pages that people see first. We'll create a beautiful landing page that introduces AI Learning Lab, a login page with OAuth buttons for Google and GitHub sign-in, implement the authentication callbacks, and create the initial dashboard layout that users see after logging in. This is the first impression of our product, so we'll make it clean and inviting using our Shadcn/ui components and Tailwind styling.

- [ ] **3.1 Create landing page UI**

**What we're doing:** Creating the public homepage ( `/` ) that introduces AI Learning Lab to visitors. This page will have a hero section explaining the product and a call-to-action button to get started.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the landing page at src/app/page.tsx:

Replace the existing content with a clean, minimalist landing page:

import Link from "next/link"

import { Button } from "@/components/ui/button"

export default function LandingPage() {

  return (

{/ *Hero Section* /}

Learn technical topics.
Daily.

A hyper-personal learning system that adapts to your goals, time, and pace.

Build real skills through structured daily sessions.

Get Started

{/ *Features Section* /}

Personalized for You

Answer a few questions and get a learning path tailored to your background, goals, and daily time budget.

One Concept Daily

Short, focused sessions (10-30 min) that fit into real life.

No overwhelming courses or endless videos.

Track Your Progress

See what you've learned and what you've built. Memory and proof-of-work

make progress visible.

{/ *Footer* /}

Built with Claude Code

)

}

Show me the created file.

```

****Expected outcome:**** Landing page created with hero and features

****Example output:****

```

src/app/page.tsx updated:

import Link from "next/link"

import { Button } from "@/components/ui/button"

export default function LandingPage() {

return (

...

)

}

```

**Verification:** Visit `http://localhost:3000` to see landing page

- [ ] **3.2 Create login page with OAuth buttons**

**What we're doing:** Building the login page ( `/login` ) with Google and GitHub sign-in buttons. When users click these buttons, they'll be redirected to the OAuth provider to authenticate.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the login page at src/app/login/page.tsx:

import { signIn } from "@/lib/auth/auth"

import { Button } from "@/components/ui/button"

import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card"

export default function LoginPage() {

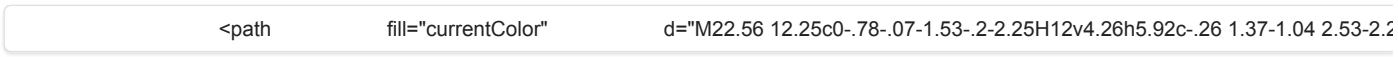
return (

Welcome to AI Learning Lab

Sign in to start your personalized learning journey

<form

```
          action={async () => {
            "use server"
            await signIn("google", { redirectTo: "/onboarding" })
          }}
        >
```




```
        <form
          action={async () => {
            "use server"
            await signIn("github", { redirectTo: "/onboarding" })
          }}
        >
```


```
  )
}
```

Show me the created file.

```

**Expected outcome:** Login page with OAuth buttons created

**Example output:**

```

src/app/login/page.tsx created:

import { signIn } from "@/lib/auth/auth"

import { Button } from "@/components/ui/button"

...

```

**Verification:** Visit `/login` to see OAuth buttons

- [ ] **3.3 Setup OAuth credentials for testing**
```

What we're doing: Getting OAuth credentials from Google Cloud Console and GitHub Settings so we can test login functionality. You'll need to create OAuth apps on both platforms.

Tool: Manual (with Claude Code guidance)

Prompt to Claude:

```

Guide me through setting up OAuth credentials:

For Google OAuth:

1. Go to https://console.cloud.google.com

2. Create a new project (or use existing)

3. Enable Google+ API

4. Go to "Credentials" > "Create Credentials" > "OAuth 2.0 Client ID"

5. Application type: Web application

6. Authorized redirect URIs: http://localhost:3000/api/auth/callback/google

7. Copy Client ID and Client Secret

For GitHub OAuth:

8. Go to https://github.com/settings/developers

9. Click "New OAuth App"

10. Application name: AI Learning Lab (Dev)

11. Homepage URL: http://localhost:3000

12. Authorization callback URL: http://localhost:3000/api/auth/callback/github

13. Copy Client ID and Client Secret

Once you have the credentials, I'll help you update the .env file.

Update .env with:

GOOGLE_CLIENT_ID=your-google-client-id-here

GOOGLE_CLIENT_SECRET=your-google-client-secret-here

GITHUB_CLIENT_ID=your-github-client-id-here

GITHUB_CLIENT_SECRET=your-github-client-secret-here

Also generate a random secret for NEXTAUTH_SECRET:

NEXTAUTH_SECRET=$(openssl rand -base64 32)

Tell me when you're ready and I'll verify the .env file is correct.

```

Expected outcome: OAuth credentials configured in .env

Example output:

```
```

.env updated with OAuth credentials:

GOOGLE_CLIENT_ID=123456789-abc...

GOOGLE_CLIENT_SECRET=GOCSPX-xyz...

GITHUB_CLIENT_ID=Iv1.abc123...

GITHUB_CLIENT_SECRET=ghp_def456...

NEXTAUTH_SECRET=randombase64string...

```

**Verification:** .env file contains all OAuth credentials

- [ ] **3.4 Create dashboard layout**

**What we're doing:** Creating the main layout for authenticated users' dashboard. This includes the navigation menu and layout structure that will wrap all dashboard pages.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the dashboard layout at src/app/(dashboard)/layout.tsx:

(Note: The (dashboard) folder with parentheses creates a route group - it groups routes

but doesn't add /dashboard to the URL path)

import { auth } from "@/lib/auth/auth"

import { redirect } from "next/navigation"

import Link from "next/link"

import { Button } from "@/components/ui/button"

export default async function DashboardLayout({

  children,

}: {

  children: React.ReactNode

}) {

  const session = await auth()

  if (!session) {

    redirect("/login")

  }

  return (

    {/ Navigation /}

AI Learning Lab

Today

Memory

Settings

{session.user?.email}

{/ *Main Content* /}

{children}

)

}

Also create a temporary placeholder for Today page at src/app/(dashboard)/today/page.tsx:

export default function TodayPage() {

return (

Today's Learning

Coming soon...

)

}

Show me both files.

```

**Expected outcome:** Dashboard layout and placeholder page created

**Example output:**

```

src/app/(dashboard)/layout.tsx created

src/app/(dashboard)/today/page.tsx created

Dashboard layout includes:

- Navigation header with Today, Memory, Settings links

- User email display

- Sticky header with blur effect

```

**Verification:** Can access `/dashboard/today` after login (redirects to login if not authenticated)

- [ ] **3.5 Test complete authentication flow**

**What we're doing:** Running the development server and testing the entire login flow from landing page to dashboard to ensure OAuth is working correctly.

**Tool:** Claude Code CLI + Manual testing

**Prompt to Claude:**

```

Start the development server and test authentication:

1. Start Next.js dev server:

npm run dev

2. The server should start on http://localhost:3000

3. Now I'll test manually:

a) Visit http://localhost:3000 (landing page)

b) Click "Get Started" → should redirect to /login

c) Click "Continue with Google" → should redirect to Google OAuth

d) After Google sign-in → should redirect to /onboarding (will 404 for now, that's OK)

e) Visit /dashboard/today → should see "Today's Learning"

4. Check the database to confirm user was created:

npx prisma studio

This opens Prisma Studio in your browser where you can see the User and AuthAccount records.

Tell me if the flow works or if you encounter any errors.

```

**Expected outcome:** Authentication flow works end-to-end

**Example output:**

```

> npm run dev

▲ Next.js 14.0.4

- Local: http://localhost:3000

- ready in 1.2s

OAuth flow tested:

✓ Landing page loads

✓ Login page shows OAuth buttons

✓ Google OAuth redirects correctly

✓ User created in database

✓ Dashboard accessible after login

```

**Verification:** Can sign in with Google/GitHub and access dashboard

- [ ] **3.6 Commit login and dashboard**

**What we're doing:** Saving our authentication UI and dashboard layout.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Commit the login and dashboard UI:

git add .

git commit -m "feat: create landing page and authentication UI

- Build landing page with hero and features sections

- Create login page with Google and GitHub OAuth buttons

- Setup dashboard layout with navigation

- Implement protected routes middleware

- Test complete authentication flow

🤖 Generated with Claude Code"

git log --oneline -3

```

**Expected outcome:** Commit created

**Example output:**

```

[feature/ai-learning-lab hij7890] feat: create landing page and authentication UI

 8 files changed, 234 insertions(+)

hij7890 feat: create landing page and authentication UI

def5678 feat: setup database schema and authentication

abc1234 chore: initial project setup

```

**Verification:** Git log shows new commit

---

## 4.0 Onboarding: Topic Selection & Questionnaire

**What we're doing:** Building the complete onboarding experience that personalizes learning for each user. After logging in, users will first enter the technical topic they want to learn (like "Docker" or "Kubernetes basics"), then complete an 8-section questionnaire with 27 questions total. This questionnaire asks about their background, goals, learning preferences, time availability, and accessibility needs. We'll build a multi-step form with validation, progress indicators, and smooth transitions. All answers get saved to the UserProfile database table so the system can generate a truly personalized learning path.

- [ ] **4.1 Create topic selection page**

**What we're doing:** Building the first screen users see after logging in - a simple, clean page where they enter the technical topic they want to learn. This topic will be used to generate their personalized course.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the topic selection page at src/app/onboarding/page.tsx:

import { redirect } from "next/navigation"

import { getCurrentUser } from "@/lib/auth/utils"

import { prisma } from "@/lib/db/prisma"

import { Button } from "@/components/ui/button"

import { Input } from "@/components/ui/input"

import { Label } from "@/components/ui/label"

```
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card"

export default async function OnboardingPage() {
  const user = await getCurrentUser()
  // Check if user already has a topic (redirect to dashboard if yes)
  const existingTopic = await prisma.topic.findFirst({
    where: { userId: user.id },
  })
  if (existingTopic) {
    redirect("/dashboard/today")
  }
  async function createTopic(formData: FormData) {
    "use server"
    const topicName = formData.get("topic") as string
    if (!topicName || topicName.trim().length < 2) {
      throw new Error("Please enter a valid topic")
    }
    await prisma.topic.create({
      data: {
        userId: user.id,
        name: topicName.trim(),
        description: `Learning ${topicName}`,
      },
    })
    redirect("/onboarding/questionnaire")
  }
  return (
```

What do you want to learn?

Enter any technical topic. The AI will create a personalized learning path just for you.

```
            Topic
            <Input
              id="topic"
              name="topic"
              placeholder="e.g., Docker, Kubernetes basics, Go programming, PostgreSQL..."
              className="text-lg"
              required
              autoFocus
            />

              Examples: "Docker containers", "React hooks", "System design", "Git workflows"

          Continue to Questionnaire

  )
}
```

Show me the created file.

```

**Expected outcome:** Topic selection page created with form

**Example output:**

```typescript
// src/app/onboarding/page.tsx created
import { redirect } from "next/navigation"
import { getCurrentUser } from "@/lib/auth/utils"
...
export default async function OnboardingPage() {
```
```

```
  // Topic selection form with server action
}
```

Verification: Visit `/onboarding` after login - should see topic input form

- [ ] **4.2 Create multi-step questionnaire component**

What we're doing: Building a reusable multi-step form component that will display one section at a time with smooth transitions and a progress indicator. This component will manage form state and navigation between sections.

Tool: Claude Code CLI

Prompt to Claude:

```
Create a multi-step questionnaire component at src/components/onboarding/QuestionnaireForm.tsx:

"use client"

import { useState } from "react"

import { Button } from "@/components/ui/button"

import { Progress } from "@/components/ui/progress"

interface QuestionnaireFormProps {

  onComplete: (answers: Record<string, any>) => void

  initialAnswers?: Record<string, any>

}

export function QuestionnaireForm({ onComplete, initialAnswers = {} }: QuestionnaireFormProps) {

  const [currentSection, setCurrentSection] = useState(1)

  const [answers, setAnswers] = useState(initialAnswers)

  const totalSections = 8

  const progress = (currentSection / totalSections) * 100

  const handleNext = () => {

    if (currentSection < totalSections) {

      setCurrentSection(currentSection + 1)

    } else {

      onComplete(answers)

    }

  }

  const handleBack = () => {

    if (currentSection > 1) {

      setCurrentSection(currentSection - 1)
```

```
  }
}
const updateAnswer = (key: string, value: any) => {
  setAnswers(prev => ({ ...prev, [key]: value }))
}
return (

    {/* Progress */}


        Section {currentSection} of {totalSections}
        {Math.round(progress)}% complete


    {/* Section Content */}

      {currentSection === 1 && (

      )}
      {currentSection === 2 && (

      )}
      {/* ... sections 3-8 */}

    {/* Navigation */}

      <Button
        type="button"
        variant="outline"
        onClick={handleBack}
        disabled={currentSection === 1}
      >
        Back
```

{currentSection === totalSections ? "Complete" : "Next"}

)

}

// Placeholder section components (we'll fill these in next tasks)

function Section1({ answers, updateAnswer }: any) {

return

Section 1: Background

}

function Section2({ answers, updateAnswer }: any) {

return

Section 2: Goals

}

Show me the created component file.

```

****Expected outcome:**** Multi-step form component created with navigation

****Example output:****

```typescript

// src/components/onboarding/QuestionnaireForm.tsx

"use client"

import { useState } from "react"

...

export function QuestionnaireForm({ onComplete, initialAnswers }: ...) {

// Multi-step form with progress tracking

}

```

****Verification:**** Component compiles without errors, exports QuestionnaireForm

- [ ] ****4.3 Install missing Shadcn/ui components (Progress)****

****What we're doing:**** Adding the Progress component from Shadcn/ui that we used in the questionnaire form.

****Tool:**** Claude Code CLI

****Prompt to Claude:****
```

```

Install the Progress component from Shadcn/ui:

npx shadcn-ui@latest add progress

This will create src/components/ui/progress.tsx

Show me confirmation that the component was installed.

```

****Expected outcome:**** Progress component installed

****Example output:****

```

✔ Done. Component installed at components/ui/progress.tsx

```

****Verification:**** File exists at `src/components/ui/progress.tsx`

- [ ] ****4.4 Create questionnaire page with Section 1 (Background)****

****What we're doing:**** Creating the main questionnaire page and implementing Section 1 with all 3 questions about the user's background and baseline skills.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Create the questionnaire page at src/app/onboarding/questionnaire/page.tsx:

This page will:

1. Import QuestionnaireForm component

2. Handle form submission to save answers to database

3. Redirect to summary page when complete

Also, update the QuestionnaireForm component to implement Section 1 fully:

Section 1 - Background & Baseline (from PRD FR-1.3.1 to FR-1.3.3):

- Q1: Current role (single-select)

- Q2: Comfortable skills (multi-select)

- Q3: Prior experience with topic (single-select)

Use RadioGroup for single-select and Checkbox for multi-select from Shadcn/ui.

Create both files with full implementation of Section 1.

Show me both files.

```

****Expected outcome:**** Questionnaire page and Section 1 fully implemented

****Example output:****

```typescript

// src/app/onboarding/questionnaire/page.tsx

import { QuestionnaireForm } from "@/components/onboarding/QuestionnaireForm"

...

// Section 1 implemented with RadioGroup and Checkbox components

```

****Verification:**** Visit `/onboarding/questionnaire` - should see Section 1 questions

- [ ] ****4.5 Implement Sections 2-3 (Goals & Learning Structure)****

****What we're doing:**** Adding Section 2 (Goals & Outcomes) and Section 3 (Learning Structure) to the questionnaire with all their questions from the PRD.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Update src/components/onboarding/QuestionnaireForm.tsx to add Sections 2 and 3:

Section 2 - Goals & Outcomes (FR-1.3.4 to FR-1.3.6):

- Q1: Why learning now (multi-select, max 2)

- Q2: Desired outcomes (multi-select)

- Q3: Depth preference (single-select)

Section 3 - Learning Structure (FR-1.3.7 to FR-1.3.9):

- Q1: Learning flow preference (single-select)

- Q2: Complexity increase preference (single-select)

- Q3: Troubleshooting importance (single-select)

Implement validation:

- Section 2 Q1: Limit to maximum 2 selections

- All required questions must be answered before "Next"

Show me the updated component with Sections 2 and 3.

```

****Expected outcome:**** Sections 2-3 added with validation

****Example output:****

```typescript

function Section2({ answers, updateAnswer }: SectionProps) {

  return (
```

Goals & Outcomes

{/ *Q1: Why learning now* /}

Why do you want to learn this topic right now? (Pick up to 2)

<Checkbox

checked={answers.learningGoals?.includes("career-transition")}

onCheckedChange={(checked) => {

// Handle multi-select with max 2 limit

}}

/>

...

)

}

```

**Verification:** Navigate to Section 2 and 3 - all questions display correctly

- [ ] **4.6 Implement Sections 4-5 (Platform & Time)**

**What we're doing:** Adding Section 4 (Platform & Tooling) and Section 5 (Time & Consistency) with all questions.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Update src/components/onboarding/QuestionnaireForm.tsx to add Sections 4 and 5:

Section 4 - Platform & Tooling (FR-1.3.10 to FR-1.3.11):

- Q1: Operating system (single-select: macOS Intel, macOS Apple Silicon, Linux, Windows WSL)

- Q2: Installation comfort (single-select)

Section 5 - Time & Consistency (FR-1.3.12 to FR-1.3.14):

- Q1: Daily time budget (single-select: 10-15, 20-30, 45-60 minutes)

- Q2: Weekly time commitment (single-select: <5, 5-10, 10+ hours)

- Q3: Missed day behavior (single-select)

Use RadioGroup for all single-select questions.

Show me the updated component with Sections 4 and 5.

```

**Expected outcome:** Sections 4-5 added

**Example output:**

```typescript

function Section5({ answers, updateAnswer }: SectionProps) {

  return (

## Time & Consistency

        On most days, how much time can you realistically spend?

        <RadioGroup

          value={answers.dailyMinutes}

          onValueChange={(value) => updateAnswer("dailyMinutes", value)}

        >

          10-15 minutes

          20-30 minutes

          45-60 minutes

  )

}

```

Verification: Navigate to Sections 4-5, all questions work

- [ ] **4.7 Implement Section 6 (Learning Style & Depth)**

What we're doing: Adding Section 6 with questions about learning style and depth preferences.

Tool: Claude Code CLI

Prompt to Claude:

```

Update src/components/onboarding/QuestionnaireForm.tsx to add Section 6:

Section 6 - Learning Style & Depth (FR-1.3.15 to FR-1.3.17):

- Q1: What helps understanding (multi-select: analogies, step-by-step labs, visuals, all)

- Q2: What frustrates (single-select)

- Q3: Depth philosophy (single-select)

Show me the updated component with Section 6.

```

Expected outcome: Section 6 added

Example output:

```typescript

function Section6({ answers, updateAnswer }: SectionProps) {

  return (

## Learning Style & Depth

        What helps you understand complex systems? (Select all that apply)

          {["Analogies to familiar concepts", "Step-by-step hands-on labs", "Diagrams and visual models", "All of the above"].map((option) => (

              <Checkbox

                checked={answers.understandingHelpers?.includes(option)}

                onCheckedChange={(checked) => {

                  // Handle multi-select

                }}

              />

              {option}

          ))}

  )

}

```

Verification: Section 6 displays all 3 questions correctly

- [ ] **4.8 Implement Section 7 (Comfort & Accessibility)**

What we're doing: Adding Section 7 - the most detailed section with 7 questions about learning comfort, accessibility needs, and UI preferences.

Tool: Claude Code CLI

Prompt to Claude:

```

Update src/components/onboarding/QuestionnaireForm.tsx to add Section 7:

Section 7 - Learning Comfort & Accessibility (FR-1.3.18 to FR-1.3.24):

- Q1: Best format (multi-select, max 2)

- Q2: Audio/video preference (single-select)

- Q3: Overwhelm triggers (multi-select, max 2)

- Q4: Daily session style (single-select)

- Q5: Content order preference (single-select)

- Q6: Missed day behavior (single-select)

- Q7: UI comfort toggles (multi-select)

This is the accessibility section - make sure labels are clear and all options

are properly described.

Show me the updated component with Section 7.

```

Expected outcome: Section 7 fully implemented with all 7 questions

Example output:

```typescript

function Section7({ answers, updateAnswer }: SectionProps) {

  return (

## Learning Comfort & Accessibility

Help us create a learning experience that works best for you

{/ *Q1: Best format* /}

Which format helps you learn best? (Pick up to 2)

<Checkbox ... />Text-first explanations

<Checkbox ... />Step-by-step hands-on labs

<Checkbox ... />Diagrams and mental models

<Checkbox ... />Short video clips (≤3 min only)

<Checkbox ... />No videos — transcript/text only

{/ *Q7: UI comfort toggles* /}

UI comfort preferences (Select any that apply)

<Checkbox ... />Focus Mode (minimal distractions)

<Checkbox ... />Reduced motion

<Checkbox ... />Larger text

<Checkbox ... />High contrast / dark mode

)

}

```

Verification: Section 7 shows all 7 questions with clear labels

- [ ] **4.9 Implement Section 8 (Application & Proof-of-Work)**

What we're doing: Adding the final section (Section 8) about application preferences and proof-of-work importance.

Tool: Claude Code CLI

Prompt to Claude:

```

Update src/components/onboarding/QuestionnaireForm.tsx to add Section 8:

Section 8 - Application & Proof-of-Work (FR-1.3.25 to FR-1.3.27):

- Q1: Comfortable application types (multi-select)

- Q2: Tracking preference (single-select)

- Q3: Proof-of-work importance (single-select)

This completes all 8 sections of the questionnaire.

Show me the updated component with Section 8 complete.

```

Expected outcome: All 8 sections complete

Example output:

```typescript

function Section8({ answers, updateAnswer }: SectionProps) {

  return (

## Application & Proof-of-Work

        What kinds of application are you comfortable with? (Select all that apply)

          <Checkbox ... />Writing code/config snippets

          <Checkbox ... />Running terminal commands

          <Checkbox ... />Linking GitHub commits/PRs

          <Checkbox ... />Writing short reflections

        How important is it to answer later: "What have I actually done in this topic?"

        <RadioGroup ...>

          Nice to have

          Important

          Very important

  )

}

```

Verification: Complete questionnaire has all 8 sections, 27 questions total

- [ ] **4.10 Create API route to save questionnaire answers**

What we're doing: Creating the API endpoint that receives the completed questionnaire answers and saves them to the UserProfile table in the database.

Tool: Claude Code CLI

Prompt to Claude:

```

Create the API route at src/app/api/onboarding/profile/route.ts:

import { NextResponse } from "next/server"

import { getCurrentUser } from "@/lib/auth/utils"

import { prisma } from "@/lib/db/prisma"

import { z } from "zod"

// Define validation schema for questionnaire answers

const profileSchema = z.object({

  // Section 1: Background

  role: z.string().optional(),

  baselineSkills: z.array(z.string()).default([]),

  priorExperience: z.string().optional(),

  // Section 2: Goals

  learningGoals: z.array(z.string()).max(2).default([]),

  desiredOutcomes: z.array(z.string()).default([]),

  depthPreference: z.string().optional(),

  // Section 3: Learning Structure

  learningFlow: z.string().optional(),

  complexityIncrease: z.string().optional(),

  troubleshootingPref: z.string().optional(),

  // Section 4: Platform

  operatingSystem: z.string().optional(),

  installationComfort: z.string().optional(),

  // Section 5: Time

  dailyMinutes: z.number().optional(),

  weeklyHours: z.number().optional(),

  missedDayBehavior: z.string().optional(),
```
```

```
  // Section 6: Learning Style
  understandingHelpers: z.array(z.string()).default([]),
  frustrationTrigger: z.string().optional(),
  depthPhilosophy: z.string().optional(),
  // Section 7: Comfort & Accessibility
  preferredFormats: z.array(z.string()).max(2).default([]),
  audioVideoPreference: z.string().optional(),
  overwhelmTriggers: z.array(z.string()).max(2).default([]),
  sessionStyle: z.string().optional(),
  contentOrder: z.string().optional(),
  uiToggles: z.array(z.string()).default([]),
  // Section 8: Application
  applicationTypes: z.array(z.string()).default([]),
  trackingPreference: z.string().optional(),
  proofOfWorkImportance: z.string().optional(),
})
export async function POST(request: Request) {
  try {
    const user = await getCurrentUser()
    const body = await request.json()
    // Validate answers
    const validatedData = profileSchema.parse(body)
    // Check if profile already exists
    const existingProfile = await prisma.userProfile.findUnique({
      where: { userId: user.id },
    })
    if (existingProfile) {
      // Update existing profile
      const profile = await prisma.userProfile.update({
        where: { userId: user.id },
        data: validatedData,
      })
      return NextResponse.json({ success: true, profile })
    } else {
```

```
      // Create new profile
      const profile = await prisma.userProfile.create({
        data: {
          userId: user.id,
          ...validatedData,
        },
      })
      return NextResponse.json({ success: true, profile })
    }
  } catch (error) {
    console.error("Error saving profile:", error)
    return NextResponse.json(
      { error: "Failed to save profile" },
      { status: 500 }
    )
  }
}
```

Show me the created API route.

```
```

****Expected outcome:**** API route created to save UserProfile

****Example output:****

```typescript
// src/app/api/onboarding/profile/route.ts
import { NextResponse } from "next/server"
import { getCurrentUser } from "@/lib/auth/utils"
...
export async function POST(request: Request) {
  // Validate and save questionnaire answers to UserProfile
}
```

```

****Verification:**** API route compiles, Prisma schema matches fields

- [ ] ****4.11 Update questionnaire page to save answers****

****What we're doing:**** Connecting the questionnaire form to the API route so that when users complete all sections, their answers are saved to the database and they're redirected to the summary page.
```

Tool: Claude Code CLI

Prompt to Claude:

```
Update src/app/onboarding/questionnaire/page.tsx to handle form submission:

export default async function QuestionnairePage() {
  const user = await getCurrentUser()
  // Check if user already has a profile
  const existingProfile = await prisma.userProfile.findUnique({
    where: { userId: user.id },
  })
  async function saveAnswers(answers: Record<string, any>) {
    "use server"
    // Call API to save answers
    const response = await fetch( ${process.env.NEXT_PUBLIC_APP_URL}/api/onboarding/profile , {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(answers),
    })
    if (response.ok) {
      redirect("/onboarding/summary")
    } else {
      throw new Error("Failed to save answers")
    }
  }
  return (

      <QuestionnaireForm
        onComplete={saveAnswers}
        initialAnswers={existingProfile || {}}
      />


  )
}

Show me the updated page.
```

```

**Expected outcome:** Form submission saves to database and redirects

**Example output:**

```typescript
// src/app/onboarding/questionnaire/page.tsx updated
async function saveAnswers(answers: Record<string, any>) {
  "use server"
  // Save to database via API
  // Redirect to summary
}
```

Verification: Complete questionnaire → answers saved → redirects to /onboarding/summary

- [ ] **4.12 Add form validation and error handling**

What we're doing: Adding client-side validation to ensure required questions are answered and adding user-friendly error messages if submission fails.

Tool: Claude Code CLI

Prompt to Claude:

```
Update src/components/onboarding/QuestionnaireForm.tsx to add validation:
1. Add state for validation errors
2. Validate that required questions are answered before allowing "Next"
3. Show error messages near unanswered required fields
4. Add loading state during form submission
5. Show error toast if submission fails
Add these validation rules:
- All single-select questions: must have a value selected
- Multi-select with "max 2": must have 0-2 items selected
- Section can only advance if all required questions are answered
Install toast component if needed:
npx shadcn-ui@latest add toast
Show me the updated component with validation.
```

Expected outcome: Form validates before advancing, shows errors

Example output:
```

```typescript
export function QuestionnaireForm({ onComplete, initialAnswers }: ...) {
  const [errors, setErrors] = useState<Record<string, string>>({})
  const [isSubmitting, setIsSubmitting] = useState(false)
  const validateSection = (section: number): boolean => {
    const newErrors: Record<string, string> = {}
    if (section === 1) {
      if (!answers.role) newErrors.role = "Please select your role"
      if (!answers.priorExperience) newErrors.priorExperience = "Required"
    }
    setErrors(newErrors)
    return Object.keys(newErrors).length === 0
  }
  const handleNext = async () => {
    if (!validateSection(currentSection)) {
      return // Stop if validation fails
    }
    if (currentSection < totalSections) {
      setCurrentSection(currentSection + 1)
    } else {
      setIsSubmitting(true)
      try {
        await onComplete(answers)
      } catch (error) {
        toast.error("Failed to save answers. Please try again.")
      } finally {
        setIsSubmitting(false)
      }
    }
  }
  return (

      {/* Show validation errors */}
      {errors.role &&
```

{errors.role}

}

{isSubmitting ? "Saving..." : "Next"}

)

}

```

**Verification:** Try advancing without answering - should show validation errors

- [ ] **4.13 Test complete questionnaire flow**

**What we're doing:** Running through the entire questionnaire end-to-end to ensure all 8 sections work, validation works, and data is saved correctly.

**Tool:** Claude Code CLI + Manual Testing

**Prompt to Claude:**

```

Let's test the complete questionnaire flow:

1. Start the dev server:

npm run dev

2. I'll test manually:

a) Login with OAuth

b) Enter a topic (e.g., "Docker")

c) Complete all 8 sections of questionnaire

d) Submit the form

e) Check that UserProfile was created in database

3. Open Prisma Studio to verify data:

npx prisma studio

4. Check the UserProfile table - should have one record with all answers

Tell me if you encounter any errors during testing.

```

**Expected outcome:** Complete flow works end-to-end

**Example output:**

```

Testing questionnaire flow:

✓ Topic selection works

✓ All 8 sections display correctly

✓ Validation prevents skipping required questions

✓ Multi-select limits work (max 2 selections)

✓ Form submits successfully

✓ UserProfile created in database with all 27 answers

✓ Redirects to /onboarding/summary

```

**Verification:** Check Prisma Studio - UserProfile table has complete record

- [ ] **4.14 Commit onboarding questionnaire**

**What we're doing:** Saving all the questionnaire work in a git commit.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Commit the questionnaire implementation:

git add .

git commit -m "feat: implement complete onboarding questionnaire

- Create topic selection page

- Build multi-step questionnaire form with 8 sections

- Implement all 27 questions from PRD

- Add form validation and error handling

- Create API route to save UserProfile

- Add progress indicator and navigation

- Test complete onboarding flow

🤖 Generated with Claude Code"

git log --oneline -5

```

**Expected outcome:** Commit created

**Example output:**

```

[feature/ai-learning-lab klm9012] feat: implement complete onboarding questionnaire

 12 files changed, 1247 insertions(+)

klm9012 feat: implement complete onboarding questionnaire

hij7890 feat: create landing page and authentication UI

def5678 feat: setup database schema and authentication

abc1234 chore: initial project setup

```

**Verification:** `git log` shows new commit

---

## 5.0 Onboarding: Learning Contract Summary (Groq API)

**What we're doing:** Integrating Groq API to generate a personalized "learning contract" summary based on the user's questionnaire answers. This summary reflects back their profile, explains how their course will be designed, and gives them a chance to confirm or correct it before learning begins. We'll create the Groq client, implement the prompt template from the PRD, build the API route, create the summary review UI, and handle the confirm/edit flow. This is our first AI integration and a critical moment of trust-building with the user.

- [ ] **5.1 Setup Groq API client**

**What we're doing:** Creating the Groq SDK client configuration and helper functions that we'll use throughout the app for all AI integrations.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the Groq client at src/lib/groq/client.ts:

import Groq from "groq-sdk"

if (!process.env.GROQ_API_KEY) {

  throw new Error("GROQ_API_KEY environment variable is not set")

}

export const groq = new Groq({

  apiKey: process.env.GROQ_API_KEY,

})

export const MODELS = {

  reasoning: "llama-3.1-70b-versatile", // For complex generation tasks

  fast: "mixtral-8x7b-instant-v0.1",   // For quick responses

}

// Helper function with retry logic

export async function callGroqWithRetry(

  apiCall: () => Promise,

  maxRetries = 3

): Promise {

  let lastError: any

  for (let attempt = 1; attempt <= maxRetries; attempt++) {

```
    try {
      return await apiCall()
    } catch (error: any) {
      lastError = error
      // Don't retry on 4xx errors (bad request, auth issues)
      if (error.status >= 400 && error.status < 500) {
        throw error
      }
      // Exponential backoff for 5xx and network errors
      if (attempt < maxRetries) {
        const delay = Math.pow(2, attempt) * 1000 // 2s, 4s, 8s
        await new Promise(resolve => setTimeout(resolve, delay))
        console.log( Retrying Groq API call (attempt ${attempt + 1}/${maxRetries}) )
      }
    }
  }
  throw lastError
}
Show me the created file.
```

****Expected outcome:**** Groq client configured with retry logic

****Example output:****

```typescript
// src/lib/groq/client.ts
import Groq from "groq-sdk"
export const groq = new Groq({ apiKey: process.env.GROQ_API_KEY })
export const MODELS = {
  reasoning: "llama-3.1-70b-versatile",
  fast: "mixtral-8x7b-instant-v0.1",
}
export async function callGroqWithRetry(...) { ... }
```

****Verification:**** File exists, imports work, GROQ_API_KEY in .env

☐ **5.2 Create learning contract prompt template**

What we're doing: Creating the prompt template file with the learning contract generation prompt from the PRD (section 9.1). This prompt will transform questionnaire answers into a personalized summary.

Tool: Claude Code CLI

Prompt to Claude:

```
Create prompt templates at src/lib/groq/prompts.ts:

import type { UserProfile } from "@prisma/client"

export function createLearningContractPrompt(profile: UserProfile, topicName: string): string {

  return `You are an expert learning designer. Based on the following learner profile, generate a clear, reflective learning contract summary.

LEARNER PROFILE:

- Role: ${profile.role || "Not specified"}

- Baseline Skills: ${profile.baselineSkills.join(", ") || "None listed"}

- Prior Experience with ${topicName}: ${profile.priorExperience || "Never used"}

- Learning Goals: ${profile.learningGoals.join(", ") || "Not specified"}

- Desired Outcomes: ${profile.desiredOutcomes.join(", ") || "Not specified"}

- Daily Time Budget: ${profile.dailyMinutes || "Not specified"} minutes

- Weekly Commitment: ${profile.weeklyHours || "Not specified"} hours

- Learning Flow Preference: ${profile.learningFlow || "Not specified"}

- Depth Philosophy: ${profile.depthPhilosophy || "Not specified"}

- Preferred Formats: ${profile.preferredFormats.join(", ") || "Not specified"}

- Accessibility Needs: ${profile.uiToggles.join(", ") || "None"}

- Application Preference: ${profile.trackingPreference || "Not specified"}

TASK:

Generate a learning contract summary in this format:

## Who You Are

[2-3 sentences restating their background, role, and current skill level]

## What You Want to Achieve

[2-3 sentences describing their goals and desired outcomes]

## How We'll Design Your Learning Path

[3-4 bullet points explaining course design decisions:]

- Pace: [daily time budget, weekly structure]

- Depth: [simple-first vs accuracy-first, based on their philosophy]
```

- Structure: [concepts-first, build-first, or mixed]

- Application: [frequency and type of hands-on work]

Learning Comfort Defaults

[2-3 bullet points on format, accessibility, overwhelm prevention]

What We'll Emphasize

[2 bullet points on what will be prioritized based on their goals]

What We'll Skip or Minimize

[2 bullet points on what will be de-emphasized to respect time budget]

Keep the tone warm, clear, and actionable. Use "we" language (collaborative). Make it feel like a thoughtful teacher who listened carefully.`

}

Show me the created file with the prompt template.

```

**Expected outcome:** Prompt template created matching PRD section 9.1

**Example output:**

```typescript

// src/lib/groq/prompts.ts

export function createLearningContractPrompt(profile: UserProfile, topicName: string): string {

  return `You are an expert learning designer...

LEARNER PROFILE:

- Role: ${profile.role}

...

Generate a learning contract summary in this format:

## Who You Are

...`

}

```

Verification: File exists with prompt template function

- [ ] **5.3 Create learning contract generation API route**

What we're doing: Building the API endpoint that takes a user's profile, calls Groq to generate the learning contract summary, and returns it to the frontend.

Tool: Claude Code CLI

Prompt to Claude:

```

Create the API route at src/app/api/onboarding/summary/route.ts:

```
import { NextResponse } from "next/server"
import { getCurrentUser } from "@/lib/auth/utils"
import { prisma } from "@/lib/db/prisma"
import { groq, MODELS, callGroqWithRetry } from "@/lib/groq/client"
import { createLearningContractPrompt } from "@/lib/groq/prompts"
export async function POST(request: Request) {
  try {
    const user = await getCurrentUser()
    // Get user's profile
    const profile = await prisma.userProfile.findUnique({
      where: { userId: user.id },
    })
    if (!profile) {
      return NextResponse.json(
        { error: "Profile not found. Complete questionnaire first." },
        { status: 404 }
)
    }
    // Get user's topic
    const topic = await prisma.topic.findFirst({
      where: { userId: user.id },
      orderBy: { createdAt: "desc" },
    })
    if (!topic) {
      return NextResponse.json(
        { error: "Topic not found" },
        { status: 404 }
)
    }
    // Generate learning contract using Groq
    const prompt = createLearningContractPrompt(profile, topic.name)
    const summary = await callGroqWithRetry(async () => {
      const completion = await groq.chat.completions.create({
```

```
        model: MODELS.reasoning,
        messages: [{ role: "user", content: prompt }],
        temperature: 0.7,
        max_tokens: 1000,
      })
      return completion.choices[0].message.content || ""
    })
    return NextResponse.json({ summary, topic: topic.name })
  } catch (error: any) {
    console.error("Error generating summary:", error)
    return NextResponse.json(
      { error: error.message || "Failed to generate summary" },
      { status: 500 }
    )
  }
}
```

Show me the created API route.

```
```

Expected outcome: API route calls Groq and returns summary

Example output:

```typescript
// src/app/api/onboarding/summary/route.ts
export async function POST(request: Request) {
  // Get user profile and topic
  // Generate prompt
  // Call Groq API with retry logic
  // Return generated summary
}
```

```

**Verification:** API route compiles, imports work

- [ ] **5.4 Create summary review page UI**

**What we're doing:** Building the page where users see their generated learning contract summary and can confirm it or go back to edit their questionnaire answers.

**Tool:** Claude Code CLI
```

Prompt to Claude:

```
Create the summary page at src/app/onboarding/summary/page.tsx:

"use client"

import { useEffect, useState } from "react"

import { useRouter } from "next/navigation"

import { Button } from "@/components/ui/button"

import { Card, CardContent, CardDescription, CardHeader, CardTitle } from "@/components/ui/card"

import { Skeleton } from "@/components/ui/skeleton"

import ReactMarkdown from "react-markdown"

export default function SummaryPage() {

  const router = useRouter()

  const [summary, setSummary] = useState("")

  const [topicName, setTopicName] = useState("")

  const [isLoading, setIsLoading] = useState(true)

  const [error, setError] = useState("")

  useEffect(() => {

    async function fetchSummary() {

      try {

        const response = await fetch("/api/onboarding/summary", {

          method: "POST",

        })

        if (!response.ok) {

          throw new Error("Failed to generate summary")

        }

        const data = await response.json()

        setSummary(data.summary)

        setTopicName(data.topic)

      } catch (err: any) {

        setError(err.message)

      } finally {

        setIsLoading(false)

      }

    }
```

```
  fetchSummary()
}, [])
const handleConfirm = async () => {
  // TODO: Create LearningStrategy and redirect to graph generation
  router.push("/onboarding/graph")
}
const handleEdit = () => {
  router.push("/onboarding/questionnaire")
}
if (error) {
  return (
        Error
        {error}
        Return to Questionnaire
  )
}
return (
      Your Learning Contract
        Here's how we'll design your {topicName} learning path based on your answers
```

{isLoading ? (

) : (

{summary}

)}

Confirm & Continue

Edit My Answers

)

}

Also install react-markdown for rendering:

npm install react-markdown

Show me the created page.

```

**Expected outcome:** Summary page displays generated contract

**Example output:**

```typescript

// src/app/onboarding/summary/page.tsx

export default function SummaryPage() {

const [summary, setSummary] = useState("")

useEffect(() => {

// Fetch summary from API

```
  }, [])

  return (

    {summary}

    Confirm & Continue

)
}
```

****Verification:**** Visit `/onboarding/summary` - should fetch and display AI-generated summary

- [ ] ****5.5 Create LearningStrategy generation function****

****What we're doing:**** Creating a function that derives the LearningStrategy record from the user's confirmed profile. This record stores the high-level course design decisions that will guide Topic Graph and Daily Plan generation.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create a utility function at src/lib/utils/learning-strategy.ts:

import type { UserProfile } from "@prisma/client"
import { prisma } from "@/lib/db/prisma"

export async function generateLearningStrategy(
  userId: string,
  topicId: string,
  profile: UserProfile
) {
  // Derive strategy from profile answers
  // Determine start level based on prior experience
  let startLevel: "beginner" | "intermediate" | "advanced" = "beginner"
  if (profile.priorExperience === "Used professionally") {
    startLevel = "advanced"
  } else if (profile.priorExperience = "Used in CI-CD" || profile.priorExperience = "Built small things") {
    startLevel = "intermediate"
  }
  // Determine phase weighting based on goals
  const phaseWeighting = {
```

```
  fundamentals: profile.learningGoals.includes("Fundamentals") ? 40 : 25,
  application: profile.desiredOutcomes.includes("Build real apps") ? 45 : 35,
  advanced: profile.depthPreference === "devops-grade mastery" ? 30 : 20,
}
// Determine daily slice policy (always one concept per day)
const dailySlicePolicy = "one-concept-strict"
// Determine application frequency
let applicationFrequency = "optional-frequent"
if (profile.trackingPreference === "learning only") {
  applicationFrequency = "none"
} else if (profile.trackingPreference === "learning + applications + evidence links") {
  applicationFrequency = "required-daily"
}
// Determine content format policy from preferred formats
const contentFormatPolicy = {
  video: profile.preferredFormats.includes("Short video clips (≤3 min only)"),
  diagrams: profile.preferredFormats.includes("Diagrams and mental models"),
  text: profile.preferredFormats.includes("Text-first explanations") ||
        profile.preferredFormats.includes("No videos — transcript/text only"),
}
// Determine link budget based on overwhelm triggers
let linkBudgetPolicy = "moderate"
if (profile.overwhelmTriggers.includes("too many links")) {
  linkBudgetPolicy = "minimal"
}
// Create or update LearningStrategy
const strategy = await prisma.learningStrategy.upsert({
  where: {
    userId_topicId: {
      userId,
      topicId,
    },
  },
  create: {
```

```
      userId,
      topicId,
      startLevel,
      phaseWeighting,
      dailySlicePolicy,
      applicationFrequency,
      contentFormatPolicy,
      linkBudgetPolicy,
    },
    update: {
      startLevel,
      phaseWeighting,
      dailySlicePolicy,
      applicationFrequency,
      contentFormatPolicy,
      linkBudgetPolicy,
    },
  })
  return strategy
}
Show me the created function.
```

****Expected outcome:**** Function derives LearningStrategy from UserProfile

****Example output:****

```typescript
// src/lib/utils/learning-strategy.ts
export async function generateLearningStrategy(userId, topicId, profile) {
  // Analyze profile to determine:
  // - Start level (beginner/intermediate/advanced)
  // - Phase weighting (fundamentals vs application vs advanced)
  // - Application frequency
  // - Content format preferences
  // - Link budget
  const strategy = await prisma.learningStrategy.upsert({ ... })
```

```
  return strategy
}
```

****Verification:**** Function compiles, Prisma schema has LearningStrategy model with all fields

- [ ] ****5.6 Update summary page to create strategy on confirm****

****What we're doing:**** Connecting the "Confirm & Continue" button to generate the LearningStrategy record and redirect to Topic Graph generation.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Update src/app/onboarding/summary/page.tsx to call strategy generation:

Add a server action to create the strategy:

async function confirmAndContinue() {
  "use server"
  const user = await getCurrentUser()
  const profile = await prisma.userProfile.findUnique({
    where: { userId: user.id },
  })
  const topic = await prisma.topic.findFirst({
    where: { userId: user.id },
  })
  if (!profile || !topic) {
    throw new Error("Profile or topic not found")
  }
  // Generate LearningStrategy
  await generateLearningStrategy(user.id, topic.id, profile)
  redirect("/onboarding/graph")
}

Update the confirm button to use this server action.

Show me the updated page.
```

****Expected outcome:**** Confirm button creates strategy and redirects

****Example output:****

```typescript
```

```
async function confirmAndContinue() {
  "use server"
  // Create LearningStrategy from profile
  await generateLearningStrategy(user.id, topic.id, profile)
  redirect("/onboarding/graph")
}
```

```
Confirm & Continue
```

```
```

Verification: Click confirm → LearningStrategy created → redirects to /onboarding/graph

- [ ] **5.7 Test learning contract generation flow**

What we're doing: Testing the complete flow from questionnaire completion through summary generation to strategy creation.

Tool: Claude Code CLI + Manual Testing

Prompt to Claude:

```
Test the learning contract generation flow:
1. Complete the questionnaire with varied answers
2. Submit → should redirect to /onboarding/summary
3. Wait for summary to load (Groq API call)
4. Verify summary is personalized and well-formatted
5. Click "Confirm & Continue"
6. Check database for LearningStrategy record
Open Prisma Studio:
npx prisma studio
Check:
- UserProfile has all answers
- LearningStrategy was created with correct userId and topicId
- Strategy fields reflect profile answers (startLevel, phaseWeighting, etc.)
Tell me what you observe in the summary and if any errors occur.
```

Expected outcome: Complete flow works, strategy created

Example output:

```

Testing learning contract flow:

✓ Summary generated in ~3 seconds

✓ Summary includes all 6 sections (Who You Are, What You Want, etc.)

✓ Summary is personalized with user's specific goals and preferences

✓ "Edit My Answers" returns to questionnaire

✓ "Confirm & Continue" creates LearningStrategy

✓ LearningStrategy in database:

  - startLevel: "intermediate" (based on prior experience)

  - applicationFrequency: "optional-frequent"

  - contentFormatPolicy: {"video": false, "text": true}

✓ Redirects to /onboarding/graph (404 for now, expected)

```

Verification: LearningStrategy exists in database with correct values

- [ ] **5.8 Commit learning contract summary**

What we're doing: Saving the Groq integration and learning contract work.

Tool: Claude Code CLI

Prompt to Claude:

```

Commit the learning contract implementation:

git add .

git commit -m "feat: integrate Groq API for learning contract generation

- Setup Groq SDK client with retry logic

- Create learning contract prompt template

- Build API route to generate personalized summary

- Create summary review page with confirm/edit flow

- Implement LearningStrategy generation from UserProfile

- Add Markdown rendering for AI-generated content

- Test complete onboarding flow through strategy creation

🤖 Generated with Claude Code"

git log --oneline -6

```

Expected outcome: Commit created

Example output:

```

[feature/ai-learning-lab nop3456] feat: integrate Groq API for learning contract generation

 9 files changed, 487 insertions(+)

nop3456 feat: integrate Groq API for learning contract generation

klm9012 feat: implement complete onboarding questionnaire

hij7890 feat: create landing page and authentication UI

...

```

Verification: `git log` shows new commit

6.0 Topic Graph Generation & Review System

What we're doing: Building the AI-powered Topic Graph generation system. Using Groq API, we'll generate a structured learning path with 15-40 concept nodes, each with prerequisites, difficulty levels, and learning content. Users will review the generated graph as an outline (not a visual graph UI), provide feedback for regeneration if needed, and lock the graph to create an immutable version. This locked TopicGraphVersion becomes the foundation for all subsequent daily learning. This is the most complex AI generation task in the system.

- [ ] **6.1 Create Groq prompt for Topic Graph generation**

What we're doing: Creating a sophisticated Groq prompt that will generate a complete Topic Graph with 15-40 concept nodes, each containing prerequisites, difficulty levels, why-it-matters statements, common confusions, and example hooks. This prompt must enforce quality constraints to prevent generic or shallow content.

Tool: Claude Code CLI

Prompt to Claude:

```

Create a new file for the Topic Graph generation prompt:

File: src/lib/groq/prompts.ts (append to existing file)

Add a new prompt template called TOPIC_GRAPH_PROMPT with these requirements:

The prompt should:

1. Take inputs: topicName (string), userProfile (object with baselineSkills, goals, outcomes, depth preference)

2. Generate 15-40 concept nodes based on topic complexity

3. Each node must include:

   - Unique concept name (one idea only)

   - Prerequisite concept IDs (array of strings)

   - Difficulty level (beginner/intermediate/advanced)

   - whyItMatters (1 sentence, real-world relevance)
```

- commonConfusions (1-2 specific misconceptions)

- exampleHook (minimal code/config example in markdown)

- estimatedMinutes (5-15 min per concept)

4. Enforce topological ordering (prerequisites come before dependents)

5. Return JSON structure matching TopicGraphVersion schema

Quality constraints to include in the prompt:

- No generic "Introduction to X" nodes - be specific

- Prerequisites must be granular concepts, not broad topics

- Common confusions must be specific misconceptions, not generic warnings

- Example hooks must be runnable code/config snippets under 10 lines

The prompt should instruct the model to think step-by-step:

- First identify core concepts

- Then add intermediate concepts to bridge gaps

- Then add advanced concepts for depth

- Finally validate prerequisite graph is acyclic

Return TypeScript code with proper typing for the prompt template.

```

****Expected outcome:**** New TOPIC_GRAPH_PROMPT template added to prompts.ts

****Example output:****

```typescript

export const TOPIC_GRAPH_PROMPT = `You are designing a learning path for {{topicName}}.

User profile:

- Baseline skills: {{baselineSkills}}

- Goals: {{goals}}

- Desired depth: {{depthPreference}}

Generate a Topic Graph with 15-40 concept nodes...

[detailed prompt continues]

Return JSON in this exact format:

{

  "topicName": "...",

  "nodes": [

    {

      "id": "concept-1",

      "conceptName": "...",

"prerequisites": [],

...

}

]

}`;

```

Verification: File updated with new prompt template

- [ ] **6.2 Create TopicGraphService with Groq integration**

What we're doing: Building a service layer that calls Groq API with the Topic Graph prompt, handles the response, validates the graph structure (especially checking for cycles in prerequisites), and stores it as a draft TopicGraphVersion in the database.

Tool: Claude Code CLI

Prompt to Claude:

```

Create a new service file for Topic Graph generation:

File: src/lib/services/topic-graph-service.ts

Implement a class TopicGraphService with these methods:

1. generateDraftGraph(topicId: string): Promise

   - Fetch topic and user profile from database

   - Build Groq prompt using TOPIC_GRAPH_PROMPT template

   - Call Groq API (llama-3.1-70b-versatile model)

   - Parse JSON response

   - Validate graph structure:

     * Check all prerequisite IDs reference existing nodes

     * Detect cycles using topological sort algorithm

     * Verify 15-40 node count

     * Validate each node has required fields

   - Save as TopicGraphVersion with locked=false

   - Return the version

2. validateGraphStructure(nodes: ConceptNode[]): { valid: boolean; errors: string[] }

   - Implement cycle detection (use DFS or Kahn's algorithm)

   - Check for orphaned nodes (except root nodes)

   - Verify difficulty progression (beginner concepts have no advanced prerequisites)

- Return validation results

3. regenerateGraph(topicGraphVersionId: string, feedback: string): Promise
   - Load previous graph version
   - Append user feedback to Groq prompt
   - Generate new version
   - Save with incremented version number

Include proper error handling, TypeScript types, and Prisma queries.

Use the Groq client from src/lib/groq/client.ts.

```

Expected outcome: TopicGraphService created with draft generation logic

Example output:

```typescript
export class TopicGraphService {
  async generateDraftGraph(topicId: string) {
    const topic = await prisma.topic.findUnique({
      where: { id: topicId },
      include: { user: { include: { profile: true } } }
    });
    const prompt = TOPIC_GRAPH_PROMPT
      .replace('{{topicName}}', topic.name)
      .replace('{{baselineSkills}}', ...);
    const response = await groqClient.chat.completions.create({
      model: 'llama-3.1-70b-versatile',
      messages: [{ role: 'user', content: prompt }],
      temperature: 0.3,
      response_format: { type: 'json_object' }
    });
    const graphData = JSON.parse(response.choices[0].message.content);
    const validation = this.validateGraphStructure(graphData.nodes);
    if (!validation.valid) {
      throw new Error( Invalid graph: ${validation.errors.join(', ')} );
    }
    return await prisma.topicGraphVersion.create({
      data: {
```
```

```
        topicId,
        version: 1,
        lockedByUser: false,
        nodesJson: graphData.nodes
      }
    });
  }
  validateGraphStructure(nodes: ConceptNode[]) {
    // Cycle detection implementation
    // ...
  }
}
```

****Verification:**** Service file created with generation and validation methods

---

- [ ] ****6.3 Create API route to generate Topic Graph****

****What we're doing:**** Creating an API endpoint that receives a topic ID, calls TopicGraphService to generate a draft graph, and returns it to the frontend. This endpoint is called right after the user confirms their learning contract summary.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create API route for Topic Graph generation:
File: src/app/api/topics/[id]/graph/route.ts
Implement POST handler that:
1. Gets authenticated user (getServerSession)
2. Validates user owns the topic
3. Checks if a draft graph already exists for this topic
4. If exists and not locked: return existing draft
5. If exists and locked: return error "Graph already locked"
6. If not exists: call TopicGraphService.generateDraftGraph()
7. Return graph data with nodes as JSON
Also implement GET handler to retrieve existing draft graph.
Include:
```

- Proper authentication checks

- Error handling for Groq API failures

- Loading state management (graph generation takes 10-30 seconds)

- TypeScript types for request/response

Use try-catch with appropriate error messages.

Return NextResponse with proper status codes.

```

**Expected outcome:** API route created at /api/topics/[id]/graph

**Example output:**

```typescript
export async function POST(
  req: Request,
  { params }: { params: { id: string } }
) {
  try {
    const session = await getServerSession(authOptions);
    if (!session?.user?.id) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }
    const topic = await prisma.topic.findUnique({
      where: { id: params.id },
      include: { graphVersions: true }
    });
    if (topic.userId !== session.user.id) {
      return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
    }
    const existingDraft = topic.graphVersions.find(v => !v.lockedByUser);
    if (existingDraft) {
      return NextResponse.json({ graph: existingDraft });
    }
    const service = new TopicGraphService();
    const graph = await service.generateDraftGraph(params.id);
    return NextResponse.json({ graph }, { status: 201 });
  } catch (error) {
```
```

```
    console.error('Topic graph generation failed:', error);
    return NextResponse.json(
      { error: 'Failed to generate topic graph' },
      { status: 500 }
    );
  }
}
```

****Verification:**** POST /api/topics/:id/graph works and generates graph

---

- [ ] ****6.4 Create Topic Graph review UI (outline view)****

****What we're doing:**** Building a page where users can review the generated Topic Graph as an outline (not a visual graph). The UI shows concepts grouped by difficulty, with prerequisites clearly indicated. Users can expand/collapse sections and see the full learning path before locking it.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create the Topic Graph review page:

File: src/app/onboarding/graph/page.tsx

Requirements:

1. Server component that fetches the draft Topic Graph

2. Display concepts as an outline grouped by difficulty:
   - Beginner Concepts (list)
   - Intermediate Concepts (list)
   - Advanced Concepts (list)

3. For each concept, show:
   - Concept name (heading)
   - Prerequisites (if any): "Requires: [concept names]"
   - Why it matters (gray text)
   - Estimated time (badge)

4. Add expandable sections to show/hide details

5. Display total concept count and estimated total hours

6. Show two actions:
   - "Regenerate with feedback" (opens feedback textarea)
```

- "Lock and Start Learning" (primary button)

UI Design:

- Clean card-based layout (reference PRD design system)

- Use accordion components from Shadcn/ui

- Subtle visual hierarchy (Beginner → Intermediate → Advanced)

- Loading state with skeleton while graph generates

- Empty state if no graph exists yet

Include client component for the interactive accordion.

Use server actions for locking the graph.

```

**Expected outcome:** Graph review page created at /onboarding/graph

**Example output:**

```typescript

export default async function TopicGraphReview() {

  const session = await getServerSession();

  const topic = await prisma.topic.findFirst({

    where: { userId: session.user.id },

    include: { graphVersions: true },

    orderBy: { createdAt: 'desc' }

  });

  const draftGraph = topic?.graphVersions.find(v => !v.lockedByUser);

  const nodes = draftGraph?.nodesJson as ConceptNode[];

  const beginnerNodes = nodes?.filter(n => n.difficulty === 'beginner');

  const intermediateNodes = nodes?.filter(n => n.difficulty === 'intermediate');

  const advancedNodes = nodes?.filter(n => n.difficulty === 'advanced');

  return (

# Review Your Learning Path: {topic.name}

We've created a {nodes.length}-concept learning path. Review the outline below,

then lock it to start learning.

```
  );
}
```

****Verification:**** Navigate to /onboarding/graph and see outline view

---

- [ ] ****6.5 Create ConceptSection component for outline display****

****What we're doing:**** Building a reusable component that displays a group of concepts (Beginner/Intermediate/Advanced) as an accordion list. Each concept is an accordion item showing name, prerequisites, why-it-matters, and estimated time. This keeps the UI clean and scannable.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create a component for concept outline display:

File: src/components/onboarding/ConceptSection.tsx

Create a client component that:

1. Accepts props: title (string), nodes (ConceptNode[])

2. Uses Shadcn Accordion component

3. Each accordion item shows:

   - Trigger: Concept name + time badge

   - Content:

     * Prerequisites (if any): linked concept names

     * Why it matters (1 sentence)

     * Common confusions (bulleted list)

     * Example hook (code block with syntax highlighting)
```

4. Show total count in section header: "Beginner (12 concepts)"

5. Closed by default (expandable on click)

Styling:

- Use card design from Shadcn

- Badge for estimated minutes

- Monospace font for code examples

- Prerequisites shown as small pills

- Muted text for metadata

Include TypeScript interface for ConceptNode prop type.

```

****Expected outcome:**** ConceptSection component created

****Example output:****

```typescript

'use client';

import { Accordion, AccordionContent, AccordionItem, AccordionTrigger } from '@/components/ui/accordion';

import { Badge } from '@/components/ui/badge';

interface ConceptNode {

  id: string;

  conceptName: string;

  prerequisites: string[];

  difficulty: string;

  whyItMatters: string;

  commonConfusions: string[];

  exampleHook: string;

  estimatedMinutes: number;

}

export function ConceptSection({

  title,

  nodes

}: {

  title: string;

  nodes: ConceptNode[];

}) {

  if (!nodes || nodes.length === 0) return null;

```
  return (

    <div>
      <h2>{title} ({nodes.length} concepts)</h2>

      {nodes.map((node) => (
        <div key={node.conceptId} data-testid="concept-node">
          {node.conceptName}
          {node.estimatedMinutes} min

          {node.prerequisites.length > 0 && (
            <div>
              Requires:
              {node.prerequisites.join(', ')}
            </div>
)}

          <p>{node.whyItMatters}</p>
          {/* ... rest of content */}
        </div>
      ))}
    </div>
  );
}
```

Verification: Concepts render as expandable accordion items

- [ ] ****6.6 Implement "Regenerate with feedback" functionality****

****What we're doing:**** Adding the ability for users to provide feedback on the generated graph (e.g., "Add more concepts about Docker networking" or "Too many beginner concepts") and regenerate a new version. The system appends the feedback to the Groq prompt and generates a new draft, incrementing the version number.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Implement graph regeneration with feedback:

Files to create/modify:

1. src/components/onboarding/RegenerateButton.tsx (client component)

2. src/app/api/topics/[id]/graph/regenerate/route.ts (API route)

RegenerateButton component should:

- Show a button "Not quite right? Regenerate with feedback"

- On click, open a dialog with textarea for feedback

- Submit button calls API route

- Show loading state during regeneration (10-30 seconds)

- On success, refresh the page to show new graph

API route should:

- Accept topicId and feedback text

- Call TopicGraphService.regenerateGraph()

- Mark old draft as archived (add isArchived field to schema)

- Create new draft version with incremented version number

- Return new graph

Include optimistic UI updates and error handling for failed regeneration.

Use Shadcn Dialog component for the feedback modal.
```

****Expected outcome:**** Users can regenerate graph with feedback

****Example output:****

```typescript
'use client';

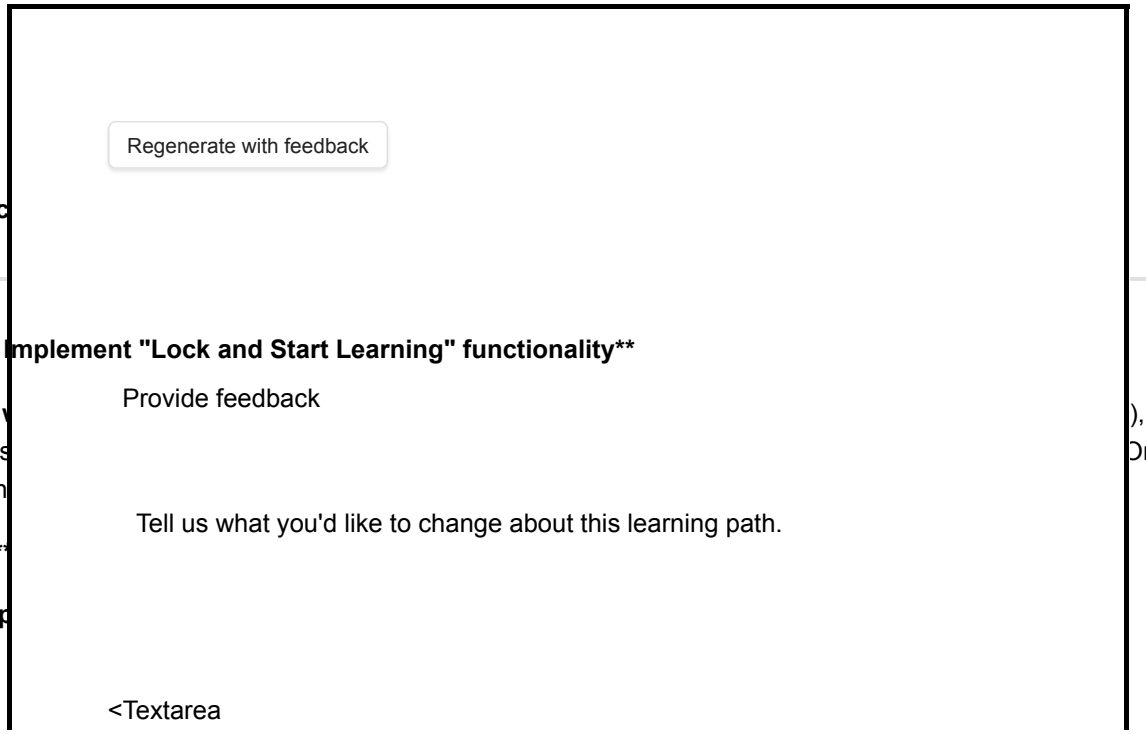
export function RegenerateButton({ topicId }: { topicId: string }) {

  const [open, setOpen] = useState(false);

  const [feedback, setFeedback] = useState('');
```

```
  const [loading, setLoading] = useState(false);
  const handleRegenerate = async () => {
    setLoading(true);
    try {
      const res = await fetch(`/api/topics/${topicId}/graph/regenerate`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ feedback })
      });
      if (res.ok) {
        router.refresh();
        setOpen(false);
        toast.success('Graph regenerated!');
      }
    } catch (error) {
      toast.error('Regeneration failed');
    } finally {
      setLoading(false);
    }
  };
  return (


  );
}
```



**Verific

- [ ] **6.7 Implement "Lock and Start Learning" functionality**

**What v), generates Once locked, th

Tool:

**Promp

```
```

Impleme

Files to

1. src/co

2. src/ap

3. src/lib

LockGra

- Primary

- On clic

"Once

- Call loc

```
            value={feedback}
            onChange={(e) => setFeedback(e.target.value)}
            placeholder="E.g., Add more concepts about networking, reduce beginner concepts..."
          />

          {loading ? 'Regenerating...' : 'Regenerate'}
```

- Redirect to /dashboard/today after success

Lock API route:

- Mark TopicGraphVersion.lockedByUser = true

- Set lockedAt timestamp

- Call DailyPlanService.generateInitialPlan()

- Return success

DailyPlanService.generateInitialPlan():

- Load locked TopicGraphVersion

- Load user's LearningStrategy (time budget, pacing)

- Sequence concepts using topological sort (prerequisites first)

- Create DailyPlan record with concept sequence

- Calculate start dates based on user's daily minutes

- Return plan

Include validation: cannot lock if graph has validation errors.

```

**Expected outcome:** Lock button works and generates daily plan

**Example output:**

```typescript
// src/app/api/topics/[id]/graph/lock/route.ts
export async function POST(req: Request, { params }: { params: { id: string } }) {
  const session = await getServerSession();
  const draftGraph = await prisma.topicGraphVersion.findFirst({
    where: {
      topicId: params.id,
```
```

```
      lockedByUser: false
    }
  });
  if (!draftGraph) {
    return NextResponse.json({ error: 'No draft graph found' }, { status: 404 });
  }
  // Lock the graph
  const lockedGraph = await prisma.topicGraphVersion.update({
    where: { id: draftGraph.id },
    data: {
      lockedByUser: true,
      lockedAt: new Date()
    }
  });
  // Generate daily plan
  const planService = new DailyPlanService();
  const plan = await planService.generateInitialPlan(lockedGraph.id, session.user.id);
  return NextResponse.json({ success: true, planId: plan.id });
}
// src/lib/services/daily-plan-service.ts
export class DailyPlanService {
  async generateInitialPlan(graphVersionId: string, userId: string) {
    const graph = await prisma.topicGraphVersion.findUnique({
      where: { id: graphVersionId },
      include: { topic: { include: { user: { include: { profile: true } } } } }
    });
    const nodes = graph.nodesJson as ConceptNode[];
    const sequence = this.topologicalSort(nodes);
    return await prisma.dailyPlan.create({
      data: {
        userId,
        topicId: graph.topicId,
        topicGraphVersionId: graphVersionId,
        conceptSequence: sequence.map(n => n.id),
```

```
        generatedAt: new Date()
      }
    });
  }
  topologicalSort(nodes: ConceptNode[]): ConceptNode[] {
    // Kahn's algorithm implementation
    // ...
  }
}
```

Verification: Locking graph creates DailyPlan and redirects to /dashboard/today

---

- [ ] **6.8 Commit Topic Graph system**

What we're doing: Committing all work related to Topic Graph generation, review, and locking.

Tool: Claude Code CLI

Prompt to Claude:

```
Create a git commit for the Topic Graph system:

git add .

git commit -m "$(cat <<'EOF'

feat: implement Topic Graph generation and review system

- Create Groq prompt for Topic Graph generation (15-40 concept nodes)

- Implement TopicGraphService with validation and cycle detection

- Add API routes for generate, regenerate, and lock

- Create outline-based review UI with accordion components

- Implement regeneration with user feedback

- Add lock mechanism with immutable versioning

- Integrate DailyPlanService to sequence concepts topologically

- Generate initial daily plan on graph lock

Topic Graph system complete. Users can now generate learning paths,

review them as outlines, regenerate with feedback, and lock to start learning.

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com
```

EOF

)"

git log --oneline -7

```

**Expected outcome:** Commit created for Topic Graph system

**Example output:**

```

[feature/ai-learning-lab qrs4567] feat: implement Topic Graph generation and review system

12 files changed, 876 insertions(+)

qrs4567 feat: implement Topic Graph generation and review system

nop3456 feat: integrate Groq API for learning contract generation

klm9012 feat: implement complete onboarding questionnaire

...

```

**Verification:** `git log` shows new commit

---

## 7.0 Daily Learning Unit (DLU) System

**What we're doing:** Building the core daily learning experience. Each day, the system serves one Daily Learning Unit containing: a concept explanation, a concrete example, reflection prompts, and an optional application moment. This is the heart of the product - where learning actually happens. The DLU must be short by design (15-20 minutes including application), respect user accessibility preferences, and create a MemoryEntry automatically on completion.

- [ ] **7.1 Create Groq prompt for DLU content generation**

**What we're doing:** Creating a prompt that generates the actual learning content for a single concept. The prompt takes a concept node (from the locked Topic Graph) and the user's learning preferences, then generates: a short concept explanation, a concrete example, 2 reflection prompts, and an optional application task. This must respect the user's format preferences (text-only, TL;DR first, etc.).

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the Daily Learning Unit content generation prompt:

File: src/lib/groq/prompts.ts (append)

Add a new prompt template called DLU_CONTENT_PROMPT with these requirements:

Inputs:

- conceptNode (object: conceptName, whyItMatters, exampleHook, commonConfusions)

- userPreferences (learningStyle, formatPreference, explanationDepth, TLDRFirst)

- prerequisitesCovered (array of completed concept names for context)

Output structure (JSON):

{

  "conceptExplanation": "...", // 200-400 words, respects user preferences

  "concreteExample": {

    "description": "...",

    "code": "...",   // markdown code block

    "stepByStep": ["step1", "step2", ...]

  },

  "reflectionPrompts": [

    "How would you explain [concept] to a teammate?",

    "What could go wrong if you [misconception]?"

  ],

  "applicationMoment": {

    "task": "...",   // 5-10 minute task

    "guidance": "...",

    "expectedOutput": "..."

  },

  "nextSteps": "..."   // 1 sentence on what comes next

}

Constraints to enforce in the prompt:

- Concept explanation must be SHORT (200-400 words max)

- If user prefers TL;DR: start with 2-sentence summary

- If user prefers text-only: no video links, only diagrams/code

- Examples must be runnable code (not pseudocode)

- Application task must be finishable in 5-10 minutes

- Reflection prompts must target common confusions

Return TypeScript code with the template.

```

**\*\*Expected outcome:\*\*** DLU_CONTENT_PROMPT added to prompts.ts

**\*\*Example output:\*\***

```typescript

export const DLU_CONTENT_PROMPT = `You are creating a Daily Learning Unit for the concept: {{conceptName}}.

User learning preferences:

- Learning style: {{learningStyle}}

- Format preference: {{formatPreference}}

- Explanation depth: {{explanationDepth}}

- TL;DR first: {{TLDRFirst}}

Prerequisites already covered: {{prerequisitesCovered}}

Generate a complete Daily Learning Unit following this structure:

1. Concept Explanation (200-400 words)

{{#if TLDRFirst}}

Start with a 2-sentence TL;DR summary, then expand.

{{/if}}

- Explain {{conceptName}} clearly and concretely

- Reference why it matters: {{whyItMatters}}

- Address common confusions: {{commonConfusions}}

{{#if formatPreference === 'text-only'}}

- Use text and diagrams only (no video links)

{{/if}}

2. Concrete Example

- Provide runnable code (not pseudocode)

- Break down step-by-step

- Keep under 20 lines of code

[... rest of prompt structure ...]

Return JSON in the exact format specified above.`;

```

Verification: New DLU prompt template exists in prompts.ts

- [ ] **7.2 Create DLUService to generate daily content**

What we're doing: Building a service that generates the content for a specific day's learning unit. It loads the concept from the DailyPlan, calls Groq with the DLU prompt, parses the response, and caches the generated content so it doesn't regenerate every time the user revisits the same day.

Tool: Claude Code CLI

Prompt to Claude:

```

Create the Daily Learning Unit service:

File: src/lib/services/dlu-service.ts

Implement a class DLUService with these methods:

1. getTodaysDLU(userId: string): Promise
   - Get user's active DailyPlan
   - Find today's concept index (based on start date + completed days)
   - Check if DLU content already generated for this concept
   - If exists: return cached content
   - If not: call generateDLUContent() and cache result
   - Return DLU object
2. generateDLUContent(conceptId: string, userId: string): Promise
   - Load concept node from TopicGraphVersion
   - Load user's learning preferences from UserProfile
   - Get list of already-completed prerequisite concepts
   - Build Groq prompt using DLU_CONTENT_PROMPT
   - Call Groq API (use llama-3.1-70b-versatile for quality)
   - Parse JSON response
   - Validate response structure
   - Cache content in database (add DLUCache model to schema)
   - Return content
3. markDayComplete(userId: string, dayIndex: number, reflectionText: string, actionTaken?: string): Promise
   - Create MemoryEntry record
   - Update DailyPlan.completedDays array
   - Return memory entry

Include:

- Proper TypeScript types for DLU and DLUContent
- Error handling for Groq failures
- Caching logic to avoid regenerating same content
- Integration with Prisma for database operations

```

Expected outcome: DLUService created with generation and completion logic

Example output:

```typescript
export class DLUService {
  async getTodaysDLU(userId: string): Promise {
    const plan = await prisma.dailyPlan.findFirst({
```
```

```
    where: { userId },
    include: { topicGraphVersion: true },
    orderBy: { generatedAt: 'desc' }
  });
  const dayIndex = this.calculateCurrentDayIndex(plan);
  const conceptId = plan.conceptSequence[dayIndex];
  // Check cache
  const cached = await prisma.dLUCache.findUnique({
    where: {
      conceptId_userId: { conceptId, userId }
    }
  });
  if (cached) {
    return {
      dayIndex,
      conceptId,
      content: cached.content as DLUContent
    };
  }
  // Generate new content
  const content = await this.generateDLUContent(conceptId, userId);
  // Cache it
  await prisma.dLUCache.create({
    data: {
      conceptId,
      userId,
      content
    }
  });
  return { dayIndex, conceptId, content };
}
async generateDLUContent(conceptId: string, userId: string) {
  const concept = await this.getConceptNode(conceptId);
  const profile = await prisma.userProfile.findUnique({ where: { userId } });
```

```
    const prompt = DLU_CONTENT_PROMPT
      .replace('{{conceptName}}', concept.conceptName)
      .replace('{{learningStyle}}', profile.learningStyle.join(', '))
      // ... more replacements
    const response = await groqClient.chat.completions.create({
      model: 'llama-3.1-70b-versatile',
      messages: [{ role: 'user', content: prompt }],
      temperature: 0.4,
      response_format: { type: 'json_object' }
    });
    const content = JSON.parse(response.choices[0].message.content);
    return content as DLUContent;
  }
  // ...
}
```

****Verification:**** DLUService methods work correctly

- [ ] ****7.3 Add DLUCache model to Prisma schema****

****What we're doing:**** Adding a database model to cache generated DLU content so we don't regenerate the same content every time a user views the same day. This improves performance and ensures consistency if the user revisits a day.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Update Prisma schema to add DLUCache model:
File: prisma/schema.prisma
Add a new model:
model DLUCache {
  id          String   @id @default(cuid())
  conceptId   String
  userId      String
  content     Json     // Stores the complete DLU content (explanation, example, prompts, application)
  generatedAt DateTime @default(now())
```

```
  user       User     @relation(fields: [userId], references: [id])
  @@unique([conceptId, userId])
  @@index([userId])
}
```

Add relation to User model:

- dluCache DLUCache[]

After updating schema, run:

```
npx prisma migrate dev --name add-dlu-cache
npx prisma generate
```

****Expected outcome:**** DLUCache model added and migration applied

****Example output:****

```
Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
✔ Enter a name for the new migration: … add-dlu-cache
Applying migration `20250125_add_dlu_cache`
The following migration(s) have been created and applied from new schema changes:
migrations/
  └─ 20250125_add_dlu_cache/
     └─ migration.sql
✔ Generated Prisma Client
```

****Verification:**** `npx prisma studio` shows DLUCache table

- [ ] ****7.4 Create API route for "Today" endpoint****

****What we're doing:**** Creating the primary API endpoint that the /dashboard/today page calls to get the current day's learning unit. This endpoint returns the concept, content, day number, and progress information.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create API route for today's Daily Learning Unit:
File: src/app/api/learning/today/route.ts
Implement GET handler that:
```

1. Gets authenticated user

2. Calls DLUService.getTodaysDLU(userId)

3. Returns:

   - dayIndex (current day number, e.g., "Day 7")

   - totalDays (total concepts in plan)

   - conceptName

   - content (DLUContent object)

   - completedToday (boolean)

4. Handle edge cases:

   - No active plan: return 404 with message "Complete onboarding first"

   - All days completed: return 200 with completion message

   - DLU generation fails: return 500 with retry option

Include proper TypeScript types and error handling.

Return NextResponse with appropriate status codes.

```

**Expected outcome:** GET /api/learning/today returns DLU data

**Example output:**

```typescript

export async function GET(req: Request) {

  try {

    const session = await getServerSession(authOptions);

    if (!session?.user?.id) {

      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

    }

    const dluService = new DLUService();

    const dlu = await dluService.getTodaysDLU(session.user.id);

    const plan = await prisma.dailyPlan.findFirst({

      where: { userId: session.user.id },

      orderBy: { generatedAt: 'desc' }

    });

    return NextResponse.json({

      dayIndex: dlu.dayIndex + 1,  // 1-indexed for display

      totalDays: plan.conceptSequence.length,

      conceptName: dlu.content.conceptName,

```
      content: dlu.content,
      completedToday: plan.completedDays.includes(dlu.dayIndex)
    });
  } catch (error) {
    console.error('Failed to load today's DLU:', error);
    return NextResponse.json(
      { error: 'Failed to load today's learning unit' },
      { status: 500 }
    );
  }
}
```

Verification: `curl http://localhost:3000/api/learning/today` returns DLU

- [ ] **7.5 Create Today page UI**

What we're doing: Building the /dashboard/today page - the most important page in the app. This is where users spend 15-20 minutes each day learning a concept, working through an example, reflecting, and optionally completing an application task. The UI must be clean, distraction-free, and match the minimalist design aesthetic.

Tool: Claude Code CLI

Prompt to Claude:

```
Create the Today page (main daily learning UI):

File: src/app/(dashboard)/today/page.tsx

Server component that:

1. Fetches today's DLU from API

2. Displays progress: "Day 7 of 32"

3. Shows concept name as page title

4. Renders four sections (each in a card):

   a) Concept Explanation (with optional TL;DR callout)

   b) Concrete Example (code block + step-by-step)

   c) Reflection Prompts (2 text inputs)

   d) Application Moment (optional, collapsible)

5. Bottom action: "Complete Day" button (primary)

6. Completed state: show checkmark + "Move to next day" button
```

Design requirements (reference PRD design system):

- Clean card-based layout with breathing room
- Generous whitespace (2rem between sections)
- Code blocks with syntax highlighting
- Muted text for metadata
- Focus mode option (hide sidebar)
- Mobile-responsive (stack cards on small screens)

Include client component for interactive reflection inputs.

Use server action for marking day complete.

```

**Expected outcome:** Today page created at /dashboard/today

**Example output:**

```typescript
export default async function TodayPage() {
  const session = await getServerSession();
  const res = await fetch( ${process.env.NEXT_PUBLIC_URL}/api/learning/today , {
    headers: { cookie: cookies().toString() }
  });
  const dlu = await res.json();
  return (

    {/ Progress indicator /}

Your learning path

## Day {dlu.dayIndex} of {dlu.totalDays}

    {/ Concept title /}

# {dlu.conceptName}

{/ *Concept explanation* /}

Concept

{dlu.content.TLDRSummary && (

TL;DR

{dlu.content.TLDRSummary}

)}

{/ *Concrete example* /}

Example

{dlu.content.concreteExample.description}

{dlu.content.concreteExample.stepByStep.map((step, i) => (

1. {i + 1}. {step}

))}

{/ *Reflection prompts* /}

{/ *Application moment* /}

{dlu.content.applicationMoment && (

)}

{/ *Complete button* /}

);

}

```

Verification: Navigate to /dashboard/today and see complete DLU

- [ ] **7.6 Create ReflectionSection component**

What we're doing: Building an interactive component for the reflection prompts. Users type their answers into text inputs, which are saved when they click "Complete Day". This component should feel lightweight and not intimidating - reflection is meant to be quick (2-3 minutes).

Tool: Claude Code CLI

Prompt to Claude:

```

Create the Reflection section component:

File: src/components/learning/ReflectionSection.tsx

Create a client component that:

1. Accepts props: prompts (string[])

2. Displays each prompt as a label above a textarea

3. Uses React state to track answers

4. Provides answers to parent component via context or callback

5. Optional: auto-save to localStorage (recover if page reloads)

Design:

- Card wrapper

- Label: muted text, slightly smaller font

- Textarea: 3-4 rows, clean border

- Character count optional (if user wants to see progress)

- No "required" enforcement - reflection is optional

Include TypeScript interface for props.

Use Shadcn Textarea component.

```

Expected outcome: ReflectionSection component created

Example output:

```typescript

'use client';

import { useState } from 'react';

import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';

import { Textarea } from '@/components/ui/textarea';

import { Label } from '@/components/ui/label';

export function ReflectionSection({

  prompts

}: {

  prompts: string[];

}) {

  const [answers, setAnswers] = useState<string[]>(Array(prompts.length).fill(''));

  const handleChange = (index: number, value: string) => {

    const newAnswers = [...answers];

    newAnswers[index] = value;

    setAnswers(newAnswers);

    // Auto-save to localStorage

    localStorage.setItem('reflection-answers', JSON.stringify(newAnswers));

  };

  return (

      Reflection

        Take a moment to think about what you just learned (optional)

```
        {prompts.map((prompt, index) => (

          {prompt}
          <Textarea
            value={answers[index]}
            onChange={(e) => handleChange(index, e.target.value)}
            placeholder="Your thoughts..."
            className="mt-2"
            rows={3}
          />

        ))}

    );
}
```

Verification: Reflection prompts render with textareas

- [ ] **7.7 Create ApplicationSection component**

What we're doing: Building the optional application moment UI. This section is collapsible (starts collapsed) and contains a task description, guidance, and expected output. Users can expand it if they want to practice, or skip it entirely. We track whether they completed the application in the MemoryEntry.

Tool: Claude Code CLI

Prompt to Claude:

```
Create the Application Moment section:

File: src/components/learning/ApplicationSection.tsx

Create a client component that:

1. Accepts props: task (ApplicationMoment object)

2. Uses Shadcn Collapsible component (starts collapsed)

3. Shows:

   - Trigger button: "Try it yourself (optional, 5-10 min)"
```

- When expanded:

* Task description

* Guidance (step-by-step hints)

* Expected output (what success looks like)

* Checkbox: "I completed this application"

4. Provides completion status to parent component

Design:

- Card with collapsible content

- Task description in regular text

- Guidance as numbered list

- Expected output in code block or muted text

- Checkbox at bottom with clear label

Include TypeScript interface for ApplicationMoment.

```

****Expected outcome:**** ApplicationSection component created

****Example output:****

```typescript

'use client';

import { useState } from 'react';

import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';

import { Collapsible, CollapsibleContent, CollapsibleTrigger } from '@/components/ui/collapsible';

import { Button } from '@/components/ui/button';

import { Checkbox } from '@/components/ui/checkbox';

import { ChevronDown } from 'lucide-react';

interface ApplicationMoment {

  task: string;

  guidance: string;

  expectedOutput: string;

}

export function ApplicationSection({

  task

}: {

  task: ApplicationMoment;

}) {

const [open, setOpen] = useState(false);

const [completed, setCompleted] = useState(false);

return (

Try it yourself (optional, 5-10 min) <ChevronDown className={ h-4 w-4 transition-transform ${open ?

## Task

{task.task}

## Guidance

{task.guidance}

## Expected output

```
{task.expectedOutput}
```

```
            <Checkbox
              id="app-complete"
              checked={completed}
              onCheckedChange={(checked) => setCompleted(checked as boolean)}
            />

              I completed this application

    );
  }
```

Verification: Application section is collapsible and tracks completion

---

- [ ] **7.8 Implement "Complete Day" functionality**

What we're doing: Creating the button and server action that marks a day as complete. When clicked, it: (1) saves reflection answers, (2) creates a MemoryEntry with the concept, reflection, and application status, (3) marks the day as completed in DailyPlan, and (4) shows a success state with option to move to next day.

Tool: Claude Code CLI

Prompt to Claude:

```
Implement the Complete Day functionality:

Files to create/modify:

1. src/components/learning/CompleteDayButton.tsx (client component)

2. src/app/actions/complete-day.ts (server action)

3. src/app/api/learning/[day]/complete/route.ts (API route)

CompleteDayButton:
```

- Primary button: "Complete Day"

- Gather reflection answers and application status from context

- Call server action

- Show loading state

- On success: show checkmark animation + "Move to Next Day" button

- On error: show toast with error message

Server action (complete-day.ts):

- Accept: dayIndex, reflectionText, applicationCompleted

- Call DLUService.markDayComplete()

- Return success/error

DLUService.markDayComplete() should:

- Create MemoryEntry with:

  * conceptId

  * reflection text (combined from prompts)

  * actionTaken (boolean from applicationCompleted)

  * timestamp

- Update DailyPlan.completedDays array (add dayIndex)

- Return memory entry

Include optimistic UI updates and confetti animation on success (optional).

```

****Expected outcome:**** Complete Day button works and creates MemoryEntry

****Example output:****

```typescript

// src/app/actions/complete-day.ts

'use server';

import { revalidatePath } from 'next/cache';

import { DLUService } from '@/lib/services/dlu-service';

export async function completeDay(

  dayIndex: number,

  reflectionText: string,

  applicationCompleted: boolean

) {

  try {

    const session = await getServerSession();

```
    if (!session?.user?.id) {
      return { error: 'Unauthorized' };
    }
    const dluService = new DLUService();
    const memoryEntry = await dluService.markDayComplete(
      session.user.id,
      dayIndex,
      reflectionText,
      applicationCompleted ? 'Completed application task' : undefined
    );
    revalidatePath('/dashboard/today');
    return { success: true, memoryEntryId: memoryEntry.id };
  } catch (error) {
    console.error('Failed to complete day:', error);
    return { error: 'Failed to complete day' };
  }
}
// src/components/learning/CompleteDayButton.tsx
'use client';
export function CompleteDayButton({ dayIndex }: { dayIndex: number }) {
  const [loading, setLoading] = useState(false);
  const [completed, setCompleted] = useState(false);
  const handleComplete = async () => {
    setLoading(true);
    const reflectionText = localStorage.getItem('reflection-answers') || '';
    const applicationCompleted = localStorage.getItem('app-completed') === 'true';
    const result = await completeDay(dayIndex, reflectionText, applicationCompleted);
    if (result.success) {
      setCompleted(true);
      localStorage.removeItem('reflection-answers');
      localStorage.removeItem('app-completed');
      confetti(); // Optional celebration
    } else {
      toast.error(result.error);
```

```
    }
    setLoading(false);
  };
  if (completed) {
    return (



Day complete! 🎉
          <Button className="mt-4" onClick={() => router.push('/dashboard/today')}>
            Move to Next Day



    );
  }
  return (
    <Button
      size="lg"
      className="mt-8 w-full"
      onClick={handleComplete}
      disabled={loading}
    >
      {loading ? 'Saving...' : 'Complete Day'}

  );
}
```

Verification: Completing day creates MemoryEntry in database

☐ **7.9 Add navigation between days**

What we're doing: Adding Previous/Next day navigation so users can review past days or skip ahead (if they want). Navigation should show which days are completed (checkmarks) and which is today (highlighted). This helps users feel oriented in their learning path.

Tool: Claude Code CLI

Prompt to Claude:

```

Add day navigation to Today page:

File: src/components/learning/DayNavigation.tsx

Create a component that:

1. Shows day navigation: [← Previous] Day 7 of 32 [Next →]

2. Previous button:

  - Disabled on Day 1

  - Navigates to /dashboard/today?day=N

3. Next button:

  - Disabled on last day

  - Enabled if current day is completed OR user wants to skip ahead

4. Optional: show mini calendar view with completed days marked

Design:

- Horizontal layout with centered day counter

- Previous/Next as icon buttons or text buttons

- Muted appearance for disabled buttons

- Optional: tooltip showing concept name on hover

Update Today page to accept `day` query parameter and load specific day's DLU.

Default to current day if no query param.

```

Expected outcome: Users can navigate between days

Example output:

```typescript

'use client';

import { useRouter, useSearchParams } from 'next/navigation';

import { Button } from '@/components/ui/button';

import { ChevronLeft, ChevronRight } from 'lucide-react';

export function DayNavigation({

  currentDay,

  totalDays,

  completedDays

}: {

  currentDay: number;
```

```
  totalDays: number;
  completedDays: number[];
}) {
  const router = useRouter();
  const goToPreviousDay = () => {
    if (currentDay > 1) {
      router.push(`/dashboard/today?day=${currentDay - 1}`);
    }
  };
  const goToNextDay = () => {
    if (currentDay < totalDays) {
      router.push(`/dashboard/today?day=${currentDay + 1}`);
    }
  };
  return (

      <Button
        variant="ghost"
        size="sm"
        onClick={goToPreviousDay}
        disabled={currentDay === 1}
      >

        Previous




Day

          {currentDay} of {totalDays}



      <Button
        variant="ghost"
```

```
        size="sm"
        onClick={goToNextDay}
        disabled={currentDay === totalDays}
      >
        Next
  );
}
```

Verification: Navigation buttons work correctly

- [ ] **7.10 Commit Daily Learning Unit system**

What we're doing: Committing all work related to the Daily Learning Unit system.

Tool: Claude Code CLI

Prompt to Claude:

```
Create a git commit for the DLU system:
git add .
git commit -m "$(cat <<'EOF'
feat: implement Daily Learning Unit (DLU) system
- Create Groq prompt for DLU content generation (respects user preferences)
- Implement DLUService with caching and completion logic
- Add DLUCache model to avoid regenerating content
- Create /dashboard/today page with clean, distraction-free UI
- Build ReflectionSection component for reflection prompts
- Build ApplicationSection component for optional practice
- Implement Complete Day functionality with MemoryEntry creation
- Add day navigation (Previous/Next buttons)
- Support query param for viewing specific days
DLU system complete. Users can now learn one concept per day,
reflect, practice, and track their progress through Memory.
```

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -8

```

**Expected outcome:** Commit created for DLU system

**Example output:**

```

[feature/ai-learning-lab tuv5678] feat: implement Daily Learning Unit (DLU) system

 14 files changed, 1243 insertions(+)

tuv5678 feat: implement Daily Learning Unit (DLU) system

qrs4567 feat: implement Topic Graph generation and review system

nop3456 feat: integrate Groq API for learning contract generation

...

```

**Verification:** `git log` shows new commit

---

## 8.0 Memory & Evidence System

**What we're doing:** Building the Memory system - the user's permanent record of what they learned and what they did. Every completed day creates a MemoryEntry. Users can view their memory timeline, filter by "applied" (entries where they completed an application), and add evidence items (code snippets, screenshots, links) to make entries "proof-backed". This is where invisible progress becomes visible.

- [ ] **8.1 Create MemoryEntry and EvidenceItem models**

**What we're doing:** Updating the Prisma schema to ensure MemoryEntry and EvidenceItem models are complete with all necessary fields. MemoryEntry stores what was learned; EvidenceItem attaches proof to a memory entry.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Verify and update Memory-related models in Prisma schema:

File: prisma/schema.prisma

Ensure these models exist with all fields:

model MemoryEntry {

  id   String   @id @default(cuid())

```
  userId         String
  topicId        String
  conceptId      String
  reflectionText String?         // User's reflection answers (combined)
  actionTaken    String?         // Description of application completed (if any)
  tags           String[]        @default([])  // topic, skill, difficulty
  createdAt      DateTime        @default(now())
  user           User            @relation(fields: [userId], references: [id])
  topic          Topic           @relation(fields: [topicId], references: [id])
  evidenceItems  EvidenceItem[]
  @@index([userId, createdAt])
  @@index([topicId])
}
model EvidenceItem {
  id             String       @id @default(cuid())
  memoryEntryId  String
  type           String       // "github_commit", "github_pr", "github_file", "link", "screenshot", "code_snippet", "output"
  urlOrBlobRef   String       // URL for links/GitHub, blob reference for uploads
  label          String?      // User-provided description
  sourceType     String       // "verifiable" (GitHub) or "self_reported" (manual)
  visibility     String       @default("private")  // "private" or "shareable"
  createdAt      DateTime     @default(now())
  memoryEntry    MemoryEntry  @relation(fields: [memoryEntryId], references: [id], onDelete: Cascade)
  @@index([memoryEntryId])
}
Add relation to User model:
- memoryEntries MemoryEntry[]
Add relation to Topic model:
- memoryEntries MemoryEntry[]
After updating schema, run:
npx prisma migrate dev --name add-memory-evidence-models
npx prisma generate
```

****Expected outcome:**** MemoryEntry and EvidenceItem models ready

Example output:

```
Applying migration `20250125_add_memory_evidence_models`

The following migration(s) have been created and applied:

migrations/
  └─ 20250125_add_memory_evidence_models/
    └─ migration.sql

✔ Generated Prisma Client
```

Verification: `npx prisma studio` shows MemoryEntry and EvidenceItem tables

- [ ] **8.2 Create Memory Timeline page**

What we're doing: Building the /dashboard/memory page - a chronological list of all MemoryEntries. This is the user's learning journal. Each entry shows the concept learned, reflection, action taken (if any), and attached evidence. Users can filter by "All", "Applied" (only entries with actionTaken), or "Proof-backed" (only entries with evidence).

Tool: Claude Code CLI

Prompt to Claude:

```
Create the Memory Timeline page:

File: src/app/(dashboard)/memory/page.tsx

Server component that:

1. Fetches all MemoryEntries for current user (newest first)

2. Displays tab filters: All | Applied | Proof-backed

3. Renders each entry as a card showing:

   - Date (relative: "2 days ago")

   - Concept name (linked to that day's DLU if user wants to revisit)

   - Reflection snippet (first 100 chars)

   - Action taken (if exists): badge "Applied"

   - Evidence count (if exists): badge "3 items"

   - Expand button to show full reflection + evidence

4. Empty state: "No memory entries yet. Complete your first day to start building your learning journal."

Design:

- Timeline layout with date markers

- Cards with subtle left border (different color for proof-backed)
```

- Applied badge in green

- Proof-backed badge in blue

- Expandable cards (accordion or modal)

Include client component for filter tabs.

Use Shadcn Tabs component.

```

**Expected outcome:** Memory timeline page created at /dashboard/memory

**Example output:**

```typescript
export default async function MemoryPage({
  searchParams
}: {
  searchParams: { filter?: string };
}) {
  const session = await getServerSession();
  const filter = searchParams.filter || 'all';
  let whereClause: any = { userId: session.user.id };
  if (filter === 'applied') {
    whereClause.actionTaken = { not: null };
  } else if (filter === 'proof-backed') {
    whereClause.evidenceItems = { some: {} };
  }
  const entries = await prisma.memoryEntry.findMany({
    where: whereClause,
    include: {
      topic: true,
      evidenceItems: true
    },
    orderBy: { createdAt: 'desc' }
  });
  return (
```
```

Memory

Your learning journal — everything you've learned and applied

{/ *Filter tabs* /}

{/ *Timeline* /}

{entries.length === 0 ? (

) : (

entries.map((entry) => (

))

)}

);

}

```

**Verification:** Navigate to /dashboard/memory and see entries

---

- [ ] **8.3 Create MemoryEntryCard component**

**What we're doing:** Building a card component that displays a single memory entry in the timeline. The card shows concept name, date, reflection snippet, action taken, and evidence count. It's expandable to show full details. This component is used in multiple places (timeline, topic view, weekly review).

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create the MemoryEntryCard component:

File: src/components/memory/MemoryEntryCard.tsx

Create a client component that:

1. Accepts props: entry (MemoryEntry with topic and evidenceItems included)

2. Displays:

- Date (formatted: "Jan 15, 2025" or "2 days ago")
- Topic badge (topic name)
- Concept name (heading)
- Reflection snippet (first 100 characters, "..." if longer)
- Applied badge (if actionTaken exists)
- Evidence count badge (if evidenceItems.length > 0)

3. Expandable to show:
- Full reflection text
- Full action taken description
- List of evidence items (clickable links)

4. Left border color:
- Default: gray
- Applied: green
- Proof-backed: blue

Design:
- Card component with hover effect
- Badge for topic name
- Applied/Evidence badges in different colors
- Expandable section using Collapsible or Accordion
- Evidence items as clickable chips

Include TypeScript interface for MemoryEntry.

Use Shadcn Card, Badge, Collapsible components.
```

****Expected outcome:**** MemoryEntryCard component created

****Example output:****

```typescript
'use client';

import { Card, CardContent, CardHeader } from '@/components/ui/card';

import { Badge } from '@/components/ui/badge';

import { Collapsible, CollapsibleContent, CollapsibleTrigger } from '@/components/ui/collapsible';

import { formatDistanceToNow } from 'date-fns';

import { ChevronDown, Check, Paperclip } from 'lucide-react';

interface MemoryEntry {
  id: string;
```
```

```
  conceptId: string;
  reflectionText: string | null;
  actionTaken: string | null;
  createdAt: Date;
  topic: { name: string };
  evidenceItems: { id: string; label: string; urlOrBlobRef: string }[];
}
export function MemoryEntryCard({ entry }: { entry: MemoryEntry }) {
  const [open, setOpen] = useState(false);
  const hasEvidence = entry.evidenceItems.length > 0;
  const hasAction = entry.actionTaken !== null;
  const borderColor = hasEvidence
    ? 'border-l-blue-500'
    : hasAction
    ? 'border-l-green-500'
    : 'border-l-gray-300';
  return (
    <Card className={`border-l-4 ${borderColor}`}>
```

```
                {entry.topic.name}
                 {formatDistanceToNow(new Date(entry.createdAt), { addSuffix: true })}
```

{entry.conceptId}

```
                {entry.reflectionText?.slice(0, 100)}
                {entry.reflectionText && entry.reflectionText.length > 100 ? '...' : ''}
```

{hasAction && (

Applied

)}

{hasEvidence && (

{entry.evidenceItems.length} {entry.evidenceItems.length === 1 ? 'item' : 'items'}

)}

```
<ChevronDown className={`h-4 w-4 transition-transform ${open ? 'rotate-180' : ''}`} />
```

{entry.reflectionText && (

Reflection

{entry.reflectionText}

)}

{entry.actionTaken && (

Action taken

{entry.actionTaken}

)}

{hasEvidence && (

Evidence

{entry.evidenceItems.map((item) => (

<a

key={item.id}

href={item.urlOrBlobRef}

target="_blank"

rel="noopener noreferrer"

className="text-sm text-blue-600 hover:underline"

>

{item.label || 'Evidence item'}

))}

)}

);

}

```
```

Verification: Memory cards render with expand/collapse

- [ ] **8.4 Create Evidence capture modal**

What we're doing: Building a modal that appears after a user completes an application moment (optional prompt: "Did this produce something you want to remember?"). The modal offers three primary actions: Import from GitHub, Paste output/link, or Skip. This must be low-friction - never force the user to curate evidence.

Tool: Claude Code CLI

Prompt to Claude:

```
Create Evidence capture modal:

File: src/components/memory/EvidenceCaptureModal.tsx

Create a client component (Dialog) that:

1. Accepts props: memoryEntryId, onComplete callback

2. Shows three action buttons:

   a) "Import from GitHub" (opens GitHub import flow)

   b) "Paste output or link" (shows textarea for manual entry)

   c) "Skip" (closes modal)

3. Manual entry form:

   - Type dropdown: Link, Code snippet, Screenshot URL, Terminal output

   - URL/Content textarea

   - Optional label input

   - Submit button

4. On submit:

   - Call API to create EvidenceItem

   - Call onComplete callback

   - Close modal

Design:

- Clean dialog with clear options

- Primary actions as large buttons (icon + text)

- Form fields appear only after selecting manual entry

- Loading state during submission

Use Shadcn Dialog, Select, Textarea components.

Include TypeScript types for evidence types.
```

Expected outcome: Evidence capture modal created

Example output:

```typescript
'use client';
import { useState } from 'react';
import { Dialog, DialogContent, DialogHeader, DialogTitle } from '@/components/ui/dialog';
import { Button } from '@/components/ui/button';
import { Textarea } from '@/components/ui/textarea';
import { Input } from '@/components/ui/input';
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from '@/components/ui/select';
import { Github, Link as LinkIcon, FileCode, Terminal } from 'lucide-react';
type EvidenceType = 'link' | 'code_snippet' | 'screenshot' | 'output';
export function EvidenceCaptureModal({
  memoryEntryId,
  open,
  onOpenChange,
  onComplete
}: {
  memoryEntryId: string;
  open: boolean;
  onOpenChange: (open: boolean) => void;
  onComplete: () => void;
}) {
  const [mode, setMode] = useState<'choose' | 'github' | 'manual'>('choose');
  const [type, setType] = useState('link');
  const [content, setContent] = useState('');
  const [label, setLabel] = useState('');
  const [loading, setLoading] = useState(false);
  const handleManualSubmit = async () => {
    setLoading(true);
    try {
      await fetch(`/api/memory/${memoryEntryId}/evidence`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
```

```
        body: JSON.stringify({
          type,
          urlOrBlobRef: content,
          label,
          sourceType: 'self_reported'
        })
      });
      onComplete();
      onOpenChange(false);
    } catch (error) {
      toast.error('Failed to add evidence');
    } finally {
      setLoading(false);
    }
  };
  return (

  );
}
```

**Verificatio

Add evidence (optional)

{mode === 'choose' && (

- [ ] **8.5 Create API route for adding evidence**

What we'r creates an EvidenceItem ype (self_reported

Tool: Cla

Prompt to

```
Create API r

File: src/app/

Implement P

1. Gets auth

2. Validates
```

<Button
  variant="outline"
  className="w-full justify-start"
  onClick={() => setMode('github')}
>
  Import from GitHub
</Button>
<Button
  variant="outline"

3. Accepts re
   - type (strin
   - urlOrBlob
   - label (stri
   - sourceTy
4. Creates E
5. Returns cr
Also impleme
Include:
- Authenticat
- Ownership
- Input valida
- Error handl
Use Zod for
```
**Expected
**Example c
```typescript
import { z } fr
const eviden
  type: z.enu
  urlOrBlobR
  label: z.strin
  sourceType
});
export asyn
  req: Reque
  { params }:
) {
  try {
    const sess
    if (!sessio
      return Ne
    }

                  className="w-full justify-start"
                  onClick={() => setMode('manual')}
                >
                  Paste output or link
              <Button
                  variant="ghost"
                  className="w-full"
                  onClick={() => onOpenChange(false)}
                >
                  Skip
            )}
            {mode === 'manual' && (
              <Select value={type} onValueChange={(v) => setType(v as EvidenceType)}>
                    Link
                    Code snippet
                    Screenshot URL
                    Terminal output
              <Textarea
                  placeholder="Paste your link, code, or output here..."
                  value={content}
                  onChange={(e) => setContent(e.target.value)}
                  rows={4}
                />

```
    const mem                <Input
      where: {                 placeholder="Label (optional)"
    });                        value={label}
    if (memory                 onChange={(e) => setLabel(e.target.value)}
      return Ne               />
    }
    const body                 <Button onClick={handleManualSubmit} disabled={!content || loading}>
    const data                   {loading ? 'Adding...' : 'Add Evidence'}
    const evid
      data: {                  <Button variant="ghost" onClick={() => setMode('choose')}>
        memory                   Back
        type: da
        urlOrBlo
        label: da
        sourceT             )}
        visibility
      }
    });
    return NextResponse.json({ evidence: evidenceItem }, { status: 201 });
  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json({ error: 'Invalid input', details: error.errors }, { status: 400 });
    }
    console.error('Failed to add evidence:', error);
    return NextResponse.json({ error: 'Failed to add evidence' }, { status: 500 });
  }
}
export async function GET(
  req: Request,
  { params }: { params: { id: string } }
) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
```

```
  }

  const memoryEntry = await prisma.memoryEntry.findUnique({

    where: { id: params.id },

    include: { evidenceItems: true }

  });

  if (memoryEntry.userId !== session.user.id) {

    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });

  }

  return NextResponse.json({ evidence: memoryEntry.evidenceItems });

}
```

Verification: Evidence items are created and attached to memory entries

---

- [ ] **8.6 Integrate evidence capture into Complete Day flow**

What we're doing: Updating the Complete Day functionality to show the evidence capture modal ONLY if the user completed an application task. After marking the day complete, if applicationCompleted is true, show the modal asking "Did this produce something you want to remember?"

Tool: Claude Code CLI

Prompt to Claude:

```
Update Complete Day flow to include evidence capture:

File: src/components/learning/CompleteDayButton.tsx (modify)

After the day is marked complete:

1. Check if applicationCompleted is true

2. If yes: show EvidenceCaptureModal

3. If no: skip evidence capture

4. Modal onComplete: show success state

Flow:

- User clicks "Complete Day"

- MemoryEntry created (as before)

- If application was completed:

  * Show evidence modal

  * User can add evidence or skip

- Show success state with "Move to Next Day" button
```

Update component to manage modal open/close state.

Import EvidenceCaptureModal component.

```

**Expected outcome:** Evidence modal appears after completing day with application

**Example output:**

```typescript
'use client';

import { EvidenceCaptureModal } from '@/components/memory/EvidenceCaptureModal';

export function CompleteDayButton({ dayIndex }: { dayIndex: number }) {
  const [loading, setLoading] = useState(false);
  const [completed, setCompleted] = useState(false);
  const [showEvidenceModal, setShowEvidenceModal] = useState(false);
  const [memoryEntryId, setMemoryEntryId] = useState<string | null>(null);
  const handleComplete = async () => {
    setLoading(true);
    const reflectionText = localStorage.getItem('reflection-answers') || '';
    const applicationCompleted = localStorage.getItem('app-completed') === 'true';
    const result = await completeDay(dayIndex, reflectionText, applicationCompleted);
    if (result.success) {
      setMemoryEntryId(result.memoryEntryId);
      if (applicationCompleted) {
        // Show evidence capture modal
        setShowEvidenceModal(true);
      } else {
        // Skip to success state
        setCompleted(true);
      }
      localStorage.removeItem('reflection-answers');
      localStorage.removeItem('app-completed');
    } else {
      toast.error(result.error);
    }
    setLoading(false);
  };
```
```

```
  const handleEvidenceComplete = () => {
    setCompleted(true);
    setShowEvidenceModal(false);
  };
  if (completed) {
    return (
```

Day complete! 🎉

```
        <Button className="mt-4" onClick={() => router.push('/dashboard/today')}>
          Move to Next Day
    );
  }
  return (
    <>
      <Button
        size="lg"
        className="mt-8 w-full"
        onClick={handleComplete}
        disabled={loading}
      >
        {loading ? 'Saving...' : 'Complete Day'}

      {memoryEntryId && (
        <EvidenceCaptureModal
          memoryEntryId={memoryEntryId}
          open={showEvidenceModal}
          onOpenChange={setShowEvidenceModal}
          onComplete={handleEvidenceComplete}
        />
      )}
```

```
    </>
  );
}
```

Verification: Completing day with application shows evidence modal

---

- [ ] **8.7 Create "Proof-backed" view in Memory page**

What we're doing: Adding a filter/view in the Memory page that shows only entries with evidence attached. This answers the question: "What have I actually done?" Users can quickly see their proof-of-work without having to scroll through all entries.

Tool: Claude Code CLI

Prompt to Claude:

```
Add Proof-backed filter to Memory page:

File: src/app/(dashboard)/memory/page.tsx (modify)

Update the filter tabs to include "Proof-backed":

- All: show all memory entries

- Applied: entries where actionTaken is not null

- Proof-backed: entries where evidenceItems.length > 0

Update the whereClause in the database query to filter correctly.

Also update MemoryFilterTabs component to show three tabs:

File: src/components/memory/MemoryFilterTabs.tsx

Create a component that renders three tabs using Shadcn Tabs:

- All

- Applied (with count badge)

- Proof-backed (with count badge)

Include counts in badges (e.g., "Applied (12)").

Active tab highlighted.

Clicking tab updates URL query param (?filter=proof-backed).
```

Expected outcome: Proof-backed filter works and shows only entries with evidence

Example output:

```typescript
// src/components/memory/MemoryFilterTabs.tsx
```

```
'use client';
import { Tabs, TabsList, TabsTrigger } from '@/components/ui/tabs';
import { useRouter, useSearchParams } from 'next/navigation';
export function MemoryFilterTabs({
  counts
}: {
  counts: { all: number; applied: number; proofBacked: number };
}) {
  const router = useRouter();
  const searchParams = useSearchParams();
  const currentFilter = searchParams.get('filter') || 'all';
  const handleFilterChange = (filter: string) => {
    router.push( /dashboard/memory?filter=${filter} );
  };
  return (
          All ({counts.all})
          Applied ({counts.applied})
          Proof-backed ({counts.proofBacked})
  );
}
// Updated memory page query
if (filter === 'proof-backed') {
  whereClause.evidenceItems = { some: {} };
}
```

```

**Verification:** Proof-backed tab shows only entries with evidence

---

- [ ] **8.8 Add topic/skill grouping view for proof-backed entries**

**What we're doing:** Creating an alternative view in the Memory page that groups proof-backed entries by topic/skill. This helps users quickly answer: "What have I done in Docker?" or "Show me all my proof-backed Kubernetes work." This is a simple grouping, not a graph UI.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Add topic grouping view to Memory page:

File: src/app/(dashboard)/memory/page.tsx (modify)

Add a fourth tab: "By Topic"

- Only shows proof-backed entries

- Groups entries by topic name

- Each topic section shows:

  * Topic name (heading)

  * Count of proof-backed entries

  * List of entries (using MemoryEntryCard component)

Update query to fetch and group by topic when this view is active.

File: src/components/memory/TopicGroupView.tsx

Create a component that:

- Accepts grouped entries (Map<string, MemoryEntry[]>)

- Renders each topic as a section

- Shows concept names and evidence counts

- Expandable to show full cards

Design:

- Accordion layout (one section per topic)

- Topic name with count badge

- Nested MemoryEntryCard components

```

**Expected outcome:** By Topic view groups proof-backed entries by topic

**Example output:**

```typescript

```
// Updated memory page with grouping
if (filter === 'by-topic') {
  const entries = await prisma.memoryEntry.findMany({
    where: {
      userId: session.user.id,
      evidenceItems: { some: {} }
    },
    include: {
      topic: true,
      evidenceItems: true
    },
    orderBy: { createdAt: 'desc' }
  });
  const grouped = entries.reduce((acc, entry) => {
    const topicName = entry.topic.name;
    if (!acc.has(topicName)) {
      acc.set(topicName, []);
    }
    acc.get(topicName)!.push(entry);
    return acc;
  }, new Map<string, typeof entries>());
  return ;
}
// src/components/memory/TopicGroupView.tsx
export function TopicGroupView({
  grouped
}: {
  grouped: Map<string, MemoryEntry[]>;
}) {
  return (

      {Array.from(grouped.entries()).map(([topicName, entries]) => (
```

```
            {topicName}

            {entries.length} {entries.length === 1 ? 'entry' : 'entries'}









            {entries.map((entry) => (


            ))}




        ))}


    );
}
```

****Verification:**** By Topic view groups entries correctly

- [ ] ****8.9 Commit Memory & Evidence system****

****What we're doing:**** Committing all work related to Memory and Evidence.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create a git commit for Memory & Evidence system:

git add .

git commit -m "$(cat <<'EOF'

feat: implement Memory and Evidence system

- Add MemoryEntry and EvidenceItem models to schema

- Create Memory Timeline page with All/Applied/Proof-backed filters

- Build MemoryEntryCard component with expand/collapse

- Create EvidenceCaptureModal for low-friction evidence capture
```

- Implement API route for adding evidence to memory entries

- Integrate evidence capture into Complete Day flow

- Add Proof-backed filter to show only entries with evidence

- Create By Topic grouping view for proof-backed entries

Memory system complete. Users can now track learning progress,

add evidence to entries, and answer "What have I actually done?"

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -9

```

**Expected outcome:** Commit created for Memory system

**Example output:**

```

[feature/ai-learning-lab wxy6789] feat: implement Memory and Evidence system

 11 files changed, 967 insertions(+)

wxy6789 feat: implement Memory and Evidence system

tuv5678 feat: implement Daily Learning Unit (DLU) system

qrs4567 feat: implement Topic Graph generation and review system

...

```

**Verification:** `git log` shows new commit

---

## 9.0 GitHub Integration for Evidence

**What we're doing:** Integrating GitHub as a verifiable evidence source. Users can connect their GitHub account (separate from login), select repositories to track, and import commits, PRs, and files as evidence items. These are marked as "verifiable" (unlike self-reported evidence) and can be linked to memory entries. This makes proof-of-work stronger and more trustworthy.

- [ ] **9.1 Add GitHubConnection model to schema**

**What we're doing:** Adding a database model to store GitHub connection details. This is separate from the GitHub OAuth used for login - this connection is specifically for evidence import, with minimal scopes requested.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Update Prisma schema to add GitHubConnection model:

File: prisma/schema.prisma

Add a new model:

```
model GitHubConnection {
  id             String   @id @default(cuid())
  userId         String   @unique
  connectedAt    DateTime @default(now())
  accessToken    String   // Encrypted token for GitHub API
  scopes         String[] @default([])  // e.g., ["repo:status", "repo:commit"]
  selectedRepos  String[] @default([])  // Repository names user wants to track
  user           User     @relation(fields: [userId], references: [id])
  @@index([userId])
}
```

Add relation to User model:

```
- githubConnection GitHubConnection?
```

After updating schema, run:

```
npx prisma migrate dev --name add-github-connection
npx prisma generate
```

****Expected outcome:**** GitHubConnection model added

****Example output:****

```
Applying migration `20250125_add_github_connection`
The following migration(s) have been created and applied:
migrations/
  └─ 20250125_add_github_connection/
     └─ migration.sql
✔ Generated Prisma Client
```

****Verification:**** `npx prisma studio` shows GitHubConnection table

- [ ] ****9.2 Create GitHub OAuth connection flow****

****What we're doing:**** Building a separate GitHub OAuth flow (different from login) specifically for connecting a GitHub account to import evidence. This requests minimal scopes (repo:status, repo:commit) and stores the access

token securely.

Tool: Claude Code CLI

Prompt to Claude:

```
Implement GitHub connection for evidence import:

Files to create:

1. src/app/api/github/connect/route.ts (OAuth initiation)

2. src/app/api/github/callback/route.ts (OAuth callback)

3. src/lib/github/client.ts (GitHub API wrapper using Octokit)

GitHub OAuth flow:

4. User clicks "Connect GitHub" in settings

5. Redirect to GitHub OAuth with minimal scopes: ["repo:status", "repo:commit"]

6. GitHub redirects back to callback route with code

7. Exchange code for access token

8. Store encrypted token in GitHubConnection table

9. Redirect to repo selection page

Use NextAuth or manual OAuth flow.

Encrypt access token before storing (use crypto library).

Include error handling for OAuth failures.

GitHub API client should:

- Use Octokit SDK

- Load token from database

- Provide methods: listRepos(), getCommits(), getPRs()
```

Expected outcome: GitHub OAuth connection flow works

Example output:

```typescript
// src/app/api/github/connect/route.ts

export async function GET(req: Request) {

  const session = await getServerSession();

  const githubAuthUrl = `https://github.com/login/oauth/authorize?client_id=${process.env.GITHUB_CLIENT_ID}&scope=repo:status,repo:commit&redirect_uri=${process.env.GITHUB_CALLBACK_URL}`;

  return NextResponse.redirect(githubAuthUrl);
```

```
}

// src/lib/github/client.ts
import { Octokit } from '@octokit/rest';

export class GitHubClient {
  private octokit: Octokit;

  constructor(accessToken: string) {
    this.octokit = new Octokit({ auth: accessToken });
  }

  async listRepos() {
    const { data } = await this.octokit.repos.listForAuthenticatedUser({
      sort: 'updated',
      per_page: 100
    });
    return data;
  }

  async getCommits(owner: string, repo: string) {
    const { data } = await this.octokit.repos.listCommits({
      owner,
      repo,
      per_page: 50
    });
    return data;
  }
}
```

****Verification:**** GitHub connection completes and stores token

---

- [ ] ****9.3 Create repository selection UI****

****What we're doing:**** Building a page where users can see their GitHub repositories and select which ones they want to track for evidence. Selected repositories are stored in GitHubConnection.selectedRepos array.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create GitHub repository selection page:
```

File: src/app/settings/github/page.tsx

Server component that:

1. Checks if GitHub is connected

2. If not connected: show "Connect GitHub" button

3. If connected: fetch user's repositories using GitHubClient

4. Display repositories as a list with checkboxes

5. Show repository name, description, and last updated date

6. Allow multi-select (checkboxes)

7. Save button to update selectedRepos

Design:

- Card-based layout

- Search/filter for repositories

- Selected repos highlighted

- Loading state while fetching repos

Include client component for checkbox interactions.

Use API route to save selected repos.

```

**Expected outcome:** Users can select repositories to track

**Example output:**

```typescript
export default async function GitHubSettings() {
  const session = await getServerSession();
  const connection = await prisma.gitHubConnection.findUnique({
    where: { userId: session.user.id }
  });
  if (!connection) {
    return (

# Connect GitHub

Connect your GitHub account to import commits and PRs as evidence.

Connect GitHub

```
    );
  }
  const githubClient = new GitHubClient(decryptToken(connection.accessToken));
  const repos = await githubClient.listRepos();
  return (

Select Repositories

Choose which repositories you want to track for evidence.

      <RepositorySelector
        repos={repos}
        selectedRepos={connection.selectedRepos}
      />

  );
}
```

Verification: Repository selection page works and saves selections

---

- [ ] **9.4 Create evidence import UI**

What we're doing: Building a modal/page that shows recent commits and PRs from selected repositories. Users can select specific commits/PRs to import as evidence items attached to a memory entry. This replaces the "Import from GitHub" option in the evidence capture modal.

Tool: Claude Code CLI

Prompt to Claude:

```
Create GitHub evidence import modal:

File: src/components/memory/GitHubImportModal.tsx

Create a client component (Dialog) that:

1. Accepts props: memoryEntryId, onComplete callback

2. Fetches recent commits/PRs from selected repos (API call)
```

3. Displays two tabs: Commits | Pull Requests

4. Each item shows:

   - Commit/PR title

   - Repository name

   - Date

   - Checkbox to select

5. Submit button to import selected items

6. Creates EvidenceItem records with type "github_commit" or "github_pr"

Design:

- Tabbed interface (Shadcn Tabs)

- List with checkboxes

- Search/filter by repo name

- Loading state while fetching

- Empty state if no repos connected

Include pagination if > 50 items.

Use API route to fetch GitHub data.

```

**Expected outcome:** Users can import commits/PRs as evidence

**Example output:**

```typescript

'use client';

export function GitHubImportModal({

  memoryEntryId,

  open,

  onOpenChange,

  onComplete

}: {

  memoryEntryId: string;

  open: boolean;

  onOpenChange: (open: boolean) => void;

  onComplete: () => void;

}) {

  const [tab, setTab] = useState<'commits' | 'prs'>('commits');

  const [items, setItems] = useState([]);

```
  const [selected, setSelected] = useState<string[]>([]);
  useEffect(() => {
    if (open) {
      fetch(`/api/github/${tab}`)
        .then(res => res.json())
        .then(data => setItems(data.items));
    }
  }, [open, tab]);
  const handleImport = async () => {
    await fetch(`/api/memory/${memoryEntryId}/evidence`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        githubItems: selected,
        type: tab === 'commits' ? 'github_commit' : 'github_pr',
        sourceType: 'verifiable'
      })
    });
    onComplete();
    onOpenChange(false);
  };
  return (

  );
}
```

Verification:

Import from GitHub

<Tabs value={tab} onValueChange={(v) => setTab(v as 'commits' | 'prs')}>

Commits

Pull Requests

- [ ] **9.5 Create API routes for fetching GitHub data**

What we're d s using the stored GitHub ac a ready for display.

Tool: Claud

Prompt to Cl

```
Create API rout
Files to create:
1. src/app/api/g
2. src/app/api/g
3. src/app/api/g
Each route sho
4. Get authentic
5. Load GitHub
6. Decrypt acce
7. Use GitHubC
8. Format and r
Commits route:
- Fetch last 50
- Sort by date (
- Return: { id, tit
PRs route:
- Fetch open +
- Return: { id, tit
Include error ha
- No GitHub co
- Invalid/expired
- GitHub API ra
```

**Expected ou

**Example out

```typescript
// src/app/api/gi
export async fu
  const session
  const connect
    where: { userId: session.user.id }
  });
  if (!connection) {
```

```
                {items.map((item) => (
                  <Checkbox
                    checked={selected.includes(item.id)}
                    onCheckedChange={(checked) => {
                      if (checked) {
                        setSelected([...selected, item.id]);
                      } else {
                        setSelected(selected.filter(id => id !== item.id));
                      }
                    }}
                  />
{item.title}
                    {item.repo} • {formatDate(item.date)}
                ))}
              <Button onClick={handleImport} disabled={selected.length === 0}>
                Import {selected.length} {selected.length === 1 ? 'item' : 'items'}
```

```
    return NextResponse.json({ error: 'GitHub not connected' }, { status: 400 });
  }
  const token = decryptToken(connection.accessToken);
  const githubClient = new GitHubClient(token);
  const allCommits = [];
  for (const repoFullName of connection.selectedRepos) {
    const [owner, repo] = repoFullName.split('/');
    const commits = await githubClient.getCommits(owner, repo);
    allCommits.push(...commits.map(c => ({
      id: c.sha,
      title: c.commit.message.split('\n')[0],
      repo: repoFullName,
      date: c.commit.author.date,
      url: c.html_url
    })));
  }
  // Sort by date descending
  allCommits.sort((a, b) => new Date(b.date).getTime() - new Date(a.date).getTime());
  return NextResponse.json({ items: allCommits.slice(0, 50) });
}
```

****Verification:**** API returns commits and PRs correctly

- [ ] ****9.6 Update EvidenceCaptureModal to include GitHub import****

****What we're doing:**** Updating the evidence capture modal to launch the GitHubImportModal when the user clicks "Import from GitHub". This connects the two modals so the flow is: Complete Day → Evidence Capture → (optional) GitHub Import.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Update EvidenceCaptureModal to integrate GitHub import:

File: src/components/memory/EvidenceCaptureModal.tsx (modify)

Changes:

1. When user clicks "Import from GitHub", set mode to 'github'
```

2. In 'github' mode, render GitHubImportModal component

3. Pass memoryEntryId and callbacks to GitHubImportModal

4. On import complete, close both modals and call onComplete

Flow:

- User clicks "Import from GitHub"

- GitHubImportModal opens (inside same dialog or nested)

- User selects commits/PRs

- Evidence items created

- Both modals close

- Success state shown

Import GitHubImportModal component.

```

**Expected outcome:** GitHub import integrated into evidence capture flow

**Example output:**

```typescript

'use client';

import { GitHubImportModal } from '@/components/memory/GitHubImportModal';

export function EvidenceCaptureModal({ ... }) {

  const [mode, setMode] = useState<'choose' | 'github' | 'manual'>('choose');

  // ... existing code ...

  return (

  );

}

```

Verification: Clicking "Import f

{mode === 'choose' && (

<Button

variant="outline"

className="w-full justify-start"

onClick={() => setMode('github')}

>

Import from GitHub

- [ ] **9.7 Commit GitHub integration system**

What we're doing: Committin nce.

Tool: Claude Code CLI

Prompt to Claude:

```

Create a git commit for GitHub int

```
git add .
git commit -m "$(cat <<'EOF'
feat: implement GitHub integration
- Add GitHubConnection model to
- Create GitHub OAuth flow with n
- Build repository selection UI in s
- Implement GitHubClient wrapper
- Create GitHubImportModal for in
- Add API routes to fetch commits/
- Integrate GitHub import into evid
- Mark GitHub-sourced evidence a
GitHub integration complete. User
select repositories, and import cor
🤖 Generated with Claude Code
Co-Authored-By: Claude Sonnet 4
EOF
)"
git log --oneline -10
```

```
{/ ... other options ... /}

)}
{mode === 'github' && (
  <GitHubImportModal
    memoryEntryId={memoryEntryId}
    open={true}
    onOpenChange={(open) => {
      if (!open) setMode('choose');
    }}
    onComplete={() => {
      onComplete();
      onOpenChange(false);
    }}
  />
)}
{/ ... manual mode ... /}
```

Expected outcome: Commit

Example output:

```
[feature/ai-learning-lab abc7890] feat: implement GitHub integration for verifiable evidence
 10 files changed, 734 insertions(+)
abc7890 feat: implement GitHub integration for verifiable evidence
wxy6789 feat: implement Memory and Evidence system
tuv5678 feat: implement Daily Learning Unit (DLU) system
...
```

Verification: `git log` shows new commit

10.0 Weekly Review & Adaptation System

What we're doing: Building the weekly review system that shows users what they accomplished, what they skipped, and how they're progressing. The system analyzes patterns (skipped days, repeated confusion, low

application rate) and suggests adjustments to pace, content format, or application frequency. Users approve or override suggestions.

- [ ] ****10.1 Create WeeklyReview Groq prompt****

****What we're doing:**** Creating a Groq prompt that analyzes a week's worth of learning data (completed days, skipped days, reflection quality, application rate) and generates: a summary of progress, insights about learning patterns, and one specific adjustment suggestion (e.g., "slow down pace" or "increase application frequency").

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create Weekly Review generation prompt:

File: src/lib/groq/prompts.ts (append)

Add a new prompt template called WEEKLY_REVIEW_PROMPT with these requirements:

Inputs:

- weekData (object containing):

  * totalDays (number of days in the week)

  * completedDays (number of days completed)

  * skippedDays (array of skipped day indices)

  * appliedDays (number of days where application was completed)

  * conceptsCovered (array of concept names)

  * averageReflectionLength (number)

  * confusionSignals (array of concepts with repeated issues)

- userProfile (original learning preferences)

Output structure (JSON):

{

  "progressSummary": "...",  // 2-3 sentences on what was accomplished

  "insights": {

    "pacing": "...",          // e.g., "You're keeping up well" or "Falling behind"

    "application": "...",     // e.g., "Applied 5 out of 7 concepts"

    "engagement": "..."       // Based on reflection quality

  },

  "adjustmentSuggestion": {

    "type": "pace_down" | "pace_up" | "more_application" | "smaller_chunks" | "none",

    "reason": "...",          // Why this adjustment is suggested

    "description": "..."      // What will change

  },
```

"nextWeekPreview": "..." // 1 sentence on what's coming

}

Constraints:

- Only suggest ONE adjustment at a time

- Be specific (not generic)

- Focus on what's working, not just problems

- Adjustment must be actionable

Return TypeScript code with the template.

```

****Expected outcome:**** WEEKLY_REVIEW_PROMPT added to prompts.ts

****Verification:**** New weekly review prompt template exists

---

- [ ] ****10.2 Create WeeklyReviewService****

****What we're doing:**** Building a service that generates weekly reviews. It loads the past week's learning data (from MemoryEntry and DailyPlan), calculates metrics, calls Groq with the weekly review prompt, and stores the review for display.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Create Weekly Review service:

File: src/lib/services/weekly-review-service.ts

Implement a class WeeklyReviewService with these methods:

1. generateWeeklyReview(userId: string): Promise

  - Get user's last 7 days of learning data

  - Calculate metrics:

    * Completed vs skipped days

    * Applied vs read-only days

    * Average reflection length

    * Concepts with repeated confusion (from reflection text analysis)

  - Build Groq prompt using WEEKLY_REVIEW_PROMPT

  - Call Groq API (use mixtral-8x7b for speed)

  - Parse JSON response

  - Return review object

2. getWeekData(userId: string): Promise

- Fetch last 7 days of MemoryEntries

- Fetch DailyPlan to see which days were skipped

- Calculate all metrics

- Return structured data

3. applyAdjustment(userId: string, adjustmentType: string): Promise

- Update user's LearningStrategy based on adjustment type

- Examples:

* "pace_down": reduce daily_minutes or increase recap frequency

* "more_application": increase application_frequency

* "smaller_chunks": flag to split future concepts

- Save changes to database

Include proper TypeScript types and error handling.

```

****Expected outcome:**** WeeklyReviewService created

****Example output:****

```typescript

export class WeeklyReviewService {

  async generateWeeklyReview(userId: string): Promise {

    const weekData = await this.getWeekData(userId);

    const profile = await prisma.userProfile.findUnique({ where: { userId } });

    const prompt = WEEKLY_REVIEW_PROMPT

      .replace('{{totalDays}}', weekData.totalDays.toString())

      .replace('{{completedDays}}', weekData.completedDays.toString())

      // ... more replacements

    const response = await groqClient.chat.completions.create({

      model: 'mixtral-8x7b-32768',

      messages: [{ role: 'user', content: prompt }],

      temperature: 0.5,

      response_format: { type: 'json_object' }

    });

    const review = JSON.parse(response.choices[0].message.content);

    return review as WeeklyReview;

  }

  async getWeekData(userId: string): Promise {

```
    const sevenDaysAgo = new Date();
    sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);
    const entries = await prisma.memoryEntry.findMany({
      where: {
        userId,
        createdAt: { gte: sevenDaysAgo }
      },
      include: { evidenceItems: true }
    });
    return {
      totalDays: 7,
      completedDays: entries.length,
      skippedDays: 7 - entries.length,
      appliedDays: entries.filter(e => e.actionTaken !== null).length,
      conceptsCovered: entries.map(e => e.conceptId),
      averageReflectionLength: entries.reduce((sum, e) => sum + (e.reflectionText?.length || 0), 0) / entries.length
    };
  }
  async applyAdjustment(userId: string, adjustmentType: string) {
    const strategy = await prisma.learningStrategy.findUnique({ where: { userId } });
    if (adjustmentType === 'pace_down') {
      await prisma.learningStrategy.update({
        where: { userId },
        data: { dailySlicePolicy: 'slower', recapFrequency: 'increased' }
      });
    }
    // ... other adjustment types
  }
}
```

Verification: WeeklyReviewService generates reviews correctly

---

- [ ] **10.3 Create Weekly Review page**

What we're doing: Building the /dashboard/review page where users see their weekly review. The page shows progress summary, insights, the adjustment suggestion (with approve/reject buttons), and a preview of next week's concepts.

Tool: Claude Code CLI

Prompt to Claude:

```
Create Weekly Review page:

File: src/app/(dashboard)/review/page.tsx

Server component that:

1. Calls WeeklyReviewService.generateWeeklyReview()

2. Displays:

   - Week date range (e.g., "Jan 15-21, 2025")

   - Progress summary (text)

   - Insights cards (pacing, application, engagement)

   - Completed vs Skipped visualization (simple chart or bars)

   - List of concepts covered this week

   - Adjustment suggestion card (if not "none"):

     * Type and reason

     * Description of what will change

     * Two buttons: "Apply" and "Keep current pace"

   - Next week preview

Design:

- Card-based layout

- Progress visualization (progress bar or simple chart)

- Adjustment suggestion highlighted (different background)

- Approve/Reject buttons clear and prominent

Include client component for adjustment approval.

Use server action for applying adjustment.
```

Expected outcome: Weekly review page created

Example output:

```typescript
export default async function WeeklyReviewPage() {

  const session = await getServerSession();
```

```
  const reviewService = new WeeklyReviewService();
  const review = await reviewService.generateWeeklyReview(session.user.id);
  return (
```

Weekly Review

```
        {getWeekDateRange()} • {review.weekData.completedDays} of 7 days completed
      {/ Progress summary /}
          This Week's Progress
{review.progressSummary}
      {/ Insights /}
      {/ Completed vs Skipped /}
          Daily Progress
          <ProgressVisualization
            completed={review.weekData.completedDays}
```

total={7}

/>

{/ *Concepts covered* /}

Concepts Covered

{review.weekData.conceptsCovered.map((concept) => (

- {concept}

))}

{/ *Adjustment suggestion* /}

{review.adjustmentSuggestion.type !== 'none' && (

)}

{/ *Next week preview* /}

Next Week

{review.nextWeekPreview}

```
  );
}
```

****Verification:**** Navigate to /dashboard/review and see weekly review

---

- [ ] ****10.4 Create AdjustmentCard component****

****What we're doing:**** Building a component that displays the adjustment suggestion with clear approve/reject actions. When approved, it calls the server action to update the user's learning strategy.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```
Create Adjustment suggestion card:

File: src/components/review/AdjustmentCard.tsx

Create a client component that:

1. Accepts props: adjustment (AdjustmentSuggestion object)

2. Displays:

   - Badge showing adjustment type (e.g., "Pace Adjustment")

   - Reason (why suggested)

   - Description (what will change)

   - Two action buttons: "Apply Adjustment" and "Keep Current Pace"

3. On "Apply", call server action to apply adjustment

4. On "Keep", dismiss card and track refusal

5. Show confirmation after applying

Design:

- Highlighted card (yellow/blue background)

- Icon for adjustment type

- Clear button labels

- Loading state during apply

- Success/dismiss animation

Use Shadcn Card, Button components.

Include server action for applying adjustment.
```

****Expected outcome:**** AdjustmentCard component created

****Example output:****

```typescript
'use client';
import { applyAdjustment } from '@/app/actions/apply-adjustment';
export function AdjustmentCard({
  adjustment
}: {
  adjustment: AdjustmentSuggestion;
}) {
  const [loading, setLoading] = useState(false);
  const [applied, setApplied] = useState(false);
  const [dismissed, setDismissed] = useState(false);
  if (dismissed || applied) return null;
  const handleApply = async () => {
    setLoading(true);
    await applyAdjustment(adjustment.type);
    setApplied(true);
    setLoading(false);
    toast.success('Adjustment applied! Your learning path will adapt accordingly.');
  };
  return (

        Suggested Adjustment

## {adjustment.reason}

{adjustment.description}
```

```
{loading ? 'Applying...' : 'Apply Adjustment'}
```

```
<Button variant="outline" onClick={() => setDismissed(true)}>
  Keep Current Pace
```

```
  );
}
```

```

**Verification:** Adjustment card shows and applies changes

---

- [ ] **10.5 Schedule weekly review generation**

**What we're doing:** Implementing automatic weekly review generation. This can be triggered weekly (via cron job or scheduled task) OR on-demand when the user visits the review page. For v1, we'll use on-demand generation (generate when user visits /dashboard/review).

**Tool:** Claude Code CLI

**Prompt to Claude:**

```
Implement weekly review scheduling:
Approach: On-demand generation (generate on page load)
Modify: src/app/(dashboard)/review/page.tsx
Changes:
1. Check if a review already exists for current week
2. If exists and not stale: show cached review
3. If not exists or stale (> 7 days old): generate new review
4. Store generated review in database for caching
Add a new model to schema:
File: prisma/schema.prisma
model WeeklyReview {
  id   String   @id @default(cuid())
  userId   String
  weekStartDate DateTime // Monday of the week
  reviewData   Json   // Stores the complete review object
```
```

generatedAt DateTime @default(now())

user User @relation(fields: [userId], references: [id])

@@unique([userId, weekStartDate])

@@index([userId])

}

Add relation to User model:

- weeklyReviews WeeklyReview[]

After updating schema, run:

npx prisma migrate dev --name add-weekly-review

npx prisma generate

Optional future enhancement: Add a cron job (Vercel Cron or similar) to pre-generate reviews every Monday.

```

**Expected outcome:** Weekly reviews are cached and reused

**Verification:** Visiting /dashboard/review generates or loads cached review

---

- [ ] **10.6 Commit Weekly Review system**

**What we're doing:** Committing all work related to weekly review and adaptation.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create a git commit for Weekly Review system:

git add .

git commit -m "$(cat <<'EOF'

feat: implement Weekly Review and Adaptation system

- Create Groq prompt for weekly review generation

- Implement WeeklyReviewService to analyze week's learning data

- Build /dashboard/review page with progress visualization

- Create AdjustmentCard component for pace/content suggestions

- Add WeeklyReview model for caching generated reviews

- Implement on-demand review generation (cached for 7 days)

- Support adjustment types: pace_down, pace_up, more_application, smaller_chunks

- Allow users to approve or reject suggested adjustments

Weekly review system complete. Users can now see weekly progress,

receive intelligent adaptation suggestions, and control their learning pace.

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -11

```

**Expected outcome:** Commit created for weekly review system

**Example output:**

```

[feature/ai-learning-lab def1234] feat: implement Weekly Review and Adaptation system

 8 files changed, 612 insertions(+)

def1234 feat: implement Weekly Review and Adaptation system

abc7890 feat: implement GitHub integration for verifiable evidence

wxy6789 feat: implement Memory and Evidence system

...

```

**Verification:** `git log` shows new commit

---

## 11.0 Settings & User Preferences

**What we're doing:** Building the settings page where users can update their learning preferences (time budget, pace, accessibility toggles, application frequency) and see their current learning strategy. Changes take effect for future daily learning units and adaptations.

- [ ] **11.1 Create Settings page layout**

**What we're doing:** Building the /dashboard/settings page with sections for: Learning Preferences, Accessibility, Application Settings, GitHub Connection, and Account. Each section is a card with form inputs.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create Settings page:

File: src/app/(dashboard)/settings/page.tsx

Server component that loads user's current preferences and displays sections:

1. Learning Preferences

   - Daily time budget (slider: 10-60 minutes)

- Weekly time commitment (slider: 1-20 hours)

- Pacing preference (radio: auto / ask before adjusting)

2. Accessibility

- Preferred format (checkboxes: text-first, diagrams, short clips, no videos)

- Order preference (radio: TL;DR first / details first / example first)

- UI toggles (checkboxes: Focus Mode, Reduced motion, Larger text, High contrast)

3. Application Settings

- Application frequency (radio: low / some / high)

- Evidence tracking (radio: learning only / with applications / with evidence)

4. GitHub Connection

- Connection status (connected/not connected)

- Button to connect/disconnect

- Link to repository selection if connected

5. Account

- Email (read-only)

- Connected accounts (Google, GitHub for login)

Design:

- Card-based sections

- Form inputs with labels

- Save button per section (or auto-save)

- Loading state on save

Include client components for form interactions.

Use server actions for saving preferences.

```

**Expected outcome:** Settings page created with all sections

**Example output:**

```typescript
export default async function SettingsPage() {
  const session = await getServerSession();
  const profile = await prisma.userProfile.findUnique({
    where: { userId: session.user.id }
  });
  return (

# Settings

{/ *Learning Preferences* /}

Learning Preferences

{/ *Accessibility* /}

Accessibility

{/ *Application Settings* /}

Application & Evidence

{/ *GitHub* /}

GitHub Connection

{/ *Account* /}

Account

);

}

```

****Verification:**** Navigate to /dashboard/settings and see all sections

---

- [ ] ****11.2 Create LearningPreferencesForm component****

****What we're doing:**** Building a form component for updating learning preferences (time budget, pacing). Uses server action to save changes to UserProfile and LearningStrategy tables.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Create Learning Preferences form:

File: src/components/settings/LearningPreferencesForm.tsx

Create a client component with form:

- Daily time budget (Slider: 10-60 min)

- Weekly time commitment (Slider: 1-20 hours)

- Pacing preference (RadioGroup: "Adapt automatically" / "Ask before adjusting")

- Save button

On submit:

- Call server action updateLearningPreferences()

- Show loading state

- Show toast on success/error

Design:

- Form with labeled inputs

- Slider shows current value

- Save button at bottom

- Auto-save option (optional)

Use Shadcn Form, Slider, RadioGroup components.

Include Zod validation.

```

Expected outcome: Learning preferences form works and saves changes

Example output:

```typescript

'use client';

import { updateLearningPreferences } from '@/app/actions/update-preferences';

export function LearningPreferencesForm({

  profile

}: {

  profile: UserProfile;

}) {

  const [dailyMinutes, setDailyMinutes] = useState(profile.dailyMinutes || 20);

  const [weeklyHours, setWeeklyHours] = useState(profile.weeklyHours || 5);

  const [pacing, setPacing] = useState(profile.pacingPreference || 'auto');

  const [loading, setLoading] = useState(false);

  const handleSave = async () => {

    setLoading(true);

    const result = await updateLearningPreferences({

      dailyMinutes,

      weeklyHours,

      pacingPreference: pacing

    });

```
  if (result.success) {
    toast.success('Preferences updated');
  } else {
    toast.error(result.error);
  }
  setLoading(false);
};
return (

      Daily time budget: {dailyMinutes} minutes
      <Slider
        value={[dailyMinutes]}
        onValueChange={([value]) => setDailyMinutes(value)}
        min={10}
        max={60}
        step={5}
        className="mt-2"
      />

      Weekly commitment: {weeklyHours} hours
      <Slider
        value={[weeklyHours]}
        onValueChange={([value]) => setWeeklyHours(value)}
        min={1}
        max={20}
        step={1}
        className="mt-2"
      />

      Pacing
```

Adapt automatically

Ask before adjusting

{loading ? 'Saving...' : 'Save Preferences'}

);

}

```

**Verification:** Form saves and updates preferences

---

- [ ] **11.3 Create AccessibilityForm component**

**What we're doing:** Building a form for updating accessibility preferences (format preferences, order, UI toggles). These preferences affect how DLU content is generated and displayed.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create Accessibility form:

File: src/components/settings/AccessibilityForm.tsx

Create a client component with form:

- Preferred formats (Checkboxes: text-first, diagrams, short clips ≤3min, no videos)

- Content order (RadioGroup: TL;DR first / details first / example first)

- UI comfort toggles (Checkboxes: Focus Mode, Reduced motion, Larger text, High contrast/dark mode)

- Save button

On submit:

- Call server action updateAccessibilityPreferences()

- Show success toast

Use Shadcn Form, Checkbox, RadioGroup components.

```

**Expected outcome:** Accessibility form works and saves changes

**Verification:** Form saves and affects future DLU generation

---

- [ ] **11.4 Create server actions for updating preferences**

**What we're doing:** Creating server actions that handle all settings updates. These actions validate input, update UserProfile and LearningStrategy tables, and return success/error responses.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create server actions for settings:

File: src/app/actions/update-preferences.ts

Implement these server actions:

1. updateLearningPreferences(data: LearningPreferencesInput)

   - Validate input with Zod

   - Update UserProfile (dailyMinutes, weeklyHours, pacingPreference)

   - Update LearningStrategy if needed

   - Return success/error

2. updateAccessibilityPreferences(data: AccessibilityInput)

   - Validate input

   - Update UserProfile (formatPreferences, orderPreference, uiToggles)

   - Return success/error

3. updateApplicationSettings(data: ApplicationInput)

   - Validate input

   - Update UserProfile (applicationFrequency, evidenceTracking)

   - Return success/error

Include proper authentication checks and error handling.

Use Zod for validation schemas.

```

**Expected outcome:** Server actions created for all settings updates

**Example output:**

```typescript

'use server';

```
import { z } from 'zod';
import { revalidatePath } from 'next/cache';
const learningPreferencesSchema = z.object({
  dailyMinutes: z.number().min(10).max(60),
  weeklyHours: z.number().min(1).max(20),
  pacingPreference: z.enum(['auto', 'ask'])
});
export async function updateLearningPreferences(
  data: z.infer
) {
  try {
    const session = await getServerSession();
    if (!session?.user?.id) {
      return { error: 'Unauthorized' };
    }
    const validated = learningPreferencesSchema.parse(data);
    await prisma.userProfile.update({
      where: { userId: session.user.id },
      data: {
        dailyMinutes: validated.dailyMinutes,
        weeklyHours: validated.weeklyHours,
        pacingPreference: validated.pacingPreference
      }
    });
    revalidatePath('/dashboard/settings');
    return { success: true };
  } catch (error) {
    if (error instanceof z.ZodError) {
      return { error: 'Invalid input', details: error.errors };
    }
    return { error: 'Failed to update preferences' };
  }
}
// ... other actions
```

```

**Verification:** Server actions update database correctly

---

- [ ] **11.5 Commit Settings system**

**What we're doing:** Committing all work related to settings and preferences.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create a git commit for Settings system:

git add .

git commit -m "$(cat <<'EOF'

feat: implement Settings and User Preferences

- Create /dashboard/settings page with sections for all preferences

- Build LearningPreferencesForm for time budget and pacing

- Build AccessibilityForm for format and UI preferences

- Create ApplicationSettingsForm for application frequency

- Implement server actions for updating all preferences

- Add validation with Zod schemas

- Support real-time preference updates (affects future DLUs)

- Include GitHub connection status in settings

Settings system complete. Users can now customize their learning

experience and update preferences at any time.

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -12

```

**Expected outcome:** Commit created for Settings system

**Example output:**

```

[feature/ai-learning-lab ghi2345] feat: implement Settings and User Preferences

 7 files changed, 523 insertions(+)

ghi2345 feat: implement Settings and User Preferences

def1234 feat: implement Weekly Review and Adaptation system

abc7890 feat: implement GitHub integration for verifiable evidence

...

```

**Verification:** `git log` shows new commit

---

## 12.0 UI Polish, Animations & Responsive Design

**What we're doing:** Polishing the entire UI to match the minimalist, personality-driven design aesthetic (reference: lelezhang.design). Adding smooth animations (Framer Motion), ensuring mobile responsiveness, implementing focus mode, and creating a cohesive visual experience across all pages.

- [ ] **12.1 Implement design system constants**

**What we're doing:** Creating a centralized design tokens file that defines colors, typography, spacing, and animation timings. This ensures consistency across all components.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create design system constants:

File: src/lib/design/tokens.ts

Define design tokens:

Colors:

- Primary: neutral grays (#FBFBFC background, `#1A1A1A` text)

- Accent: subtle blue for links/actions (#0066CC)

- Success: muted green (#059669)

- Warning: soft yellow (#F59E0B)

- Borders: `#E5E5E5`

Typography:

- Font family: system font stack (fallback to -apple-system, sans-serif)

- Sizes: xs (12px), sm (14px), base (16px), lg (18px), xl (20px), 2xl (24px), 3xl (30px)

- Weights: normal (400), medium (500), semibold (600), bold (700)

- Line heights: tight (1.25), normal (1.5), relaxed (1.75)

Spacing:

- Scale: 4px base (1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64)

- Component padding: 24px (cards), 16px (buttons)

- Section spacing: 48px (between major sections)

Animations:

- Duration: fast (150ms), normal (250ms), slow (350ms)

- Easing: ease-out (default), spring (for interactive elements)

Export as TypeScript constants.

Update Tailwind config to use these tokens.

```

**Expected outcome:** Design tokens file created and integrated

**Verification:** Tokens are used consistently across components

---

- [ ] **12.2 Add Framer Motion animations**

**What we're doing:** Integrating Framer Motion for smooth page transitions, card entrance animations, and interactive feedback. Animations should be subtle (150ms) and respect reduced motion preferences.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Add Framer Motion animations:

Install dependency:

npm install framer-motion

Create animation utilities:

File: src/lib/animations/variants.ts

Define reusable animation variants:

- fadeIn: opacity 0 → 1 (150ms)

- slideUp: translateY 20px → 0 (200ms)

- scaleIn: scale 0.95 → 1 (150ms)

- cardHover: scale 1 → 1.02 (150ms)

Apply animations to key components:

1. Page transitions (layout.tsx)

2. Card entrance (MemoryEntryCard, ConceptSection)

3. Button interactions (hover, tap)

4. Modal/dialog entrance

Respect user's reduced motion preference:

- Check prefers-reduced-motion media query

- Disable animations if user prefers reduced motion

Use AnimatePresence for exit animations.

```

**Expected outcome:** Smooth animations added throughout app

**Example output:**

```typescript
// src/lib/animations/variants.ts
export const fadeIn = {
  initial: { opacity: 0 },
  animate: { opacity: 1 },
  exit: { opacity: 0 },
  transition: { duration: 0.15 }
};
export const slideUp = {
  initial: { opacity: 0, y: 20 },
  animate: { opacity: 1, y: 0 },
  exit: { opacity: 0, y: -20 },
  transition: { duration: 0.2, ease: 'easeOut' }
};
// Usage in component
import { motion } from 'framer-motion';
import { fadeIn } from '@/lib/animations/variants';
export function MemoryEntryCard({ entry }) {
  return (
    <motion.div {...fadeIn}>
      ...
    </motion.div>
  );
}
```

Verification: Animations work and respect reduced motion

- [ ] **12.3 Implement responsive design**

What we're doing: Ensuring all pages and components work perfectly on mobile, tablet, and desktop. Using Tailwind responsive utilities (sm:, md:, lg:) to adapt layouts.
```

Tool: Claude Code CLI

Prompt to Claude:

```
Make all pages and components responsive:

Breakpoints (Tailwind defaults):

- sm: 640px (mobile landscape)

- md: 768px (tablet portrait)

- lg: 1024px (desktop)

Key responsive changes:

1. Navigation

  - Mobile: hamburger menu (drawer)

  - Desktop: sidebar or top nav

2. Card layouts

  - Mobile: single column (stack)

  - Tablet: 2 columns where appropriate

  - Desktop: 3 columns for grid layouts

3. Typography

  - Mobile: smaller headings (scale down by 20%)

  - Desktop: full scale

4. Spacing

  - Mobile: reduced padding (16px instead of 24px)

  - Desktop: full spacing

5. Daily Learning Unit page

  - Mobile: stack all sections vertically

  - Desktop: maintain card layout with generous whitespace

6. Memory Timeline

  - Mobile: simplified cards (hide some metadata)

  - Desktop: full cards with all details

Test on:

- iPhone SE (375px)

- iPad (768px)

- Desktop (1920px)

Use Chrome DevTools responsive mode for testing.
```

Expected outcome: App works on all screen sizes

Verification: Test on mobile, tablet, and desktop viewports

- [ ] **12.4 Implement Focus Mode**

What we're doing: Creating a "Focus Mode" toggle that hides the sidebar/navigation and centers content for distraction-free learning. This is especially important for the Today page.

Tool: Claude Code CLI

Prompt to Claude:

```
Implement Focus Mode:

Create context for focus mode state:

File: src/contexts/FocusModeContext.tsx

Create a React context that:

- Stores focusMode boolean state

- Provides toggleFocusMode function

- Persists state to localStorage

Update dashboard layout:

File: src/app/(dashboard)/layout.tsx

Modify to:

- Hide sidebar when focusMode is true

- Show floating focus mode toggle button (bottom-right)

- Increase max-width when in focus mode (from 4xl to 6xl)

Add focus mode toggle to Today page:

- Keyboard shortcut: F (toggle focus mode)

- Visual indicator when in focus mode

Design:

- Smooth transition (300ms) when entering/exiting focus mode

- Toggle button: subtle, bottom-right corner

- Icon: Focus/Unfocus (eye icon)

Use Framer Motion for layout transitions.
```

Expected outcome: Focus mode works and hides distractions

Verification: Toggle focus mode and verify sidebar hides

☐ **12.5 Add loading states and skeletons**

What we're doing: Creating skeleton loaders for all pages that fetch data. This improves perceived performance and provides visual feedback during loading.

Tool: Claude Code CLI

Prompt to Claude:

```
Add loading states and skeleton screens:

Create skeleton components:

File: src/components/ui/skeleton.tsx (use Shadcn skeleton component)

Create specific skeletons for:

1. DLU page (CardSkeleton with text lines)

2. Memory timeline (MemoryCardSkeleton)

3. Topic graph outline (AccordionSkeleton)

4. Weekly review (ReviewCardSkeleton)

Use Next.js loading.tsx files:

- src/app/(dashboard)/today/loading.tsx

- src/app/(dashboard)/memory/loading.tsx

- src/app/(dashboard)/review/loading.tsx

- src/app/onboarding/graph/loading.tsx

Each loading file should render appropriate skeleton components.

Design:

- Pulse animation (subtle)

- Match actual component structure

- Gray background (#E5E5E5)

Use Shadcn Skeleton component as base.
```

Expected outcome: Skeleton loaders show during data fetching

Verification: Navigate to pages and see smooth loading states

---

☐ **12.6 Polish component styling**

What we're doing: Reviewing and polishing all components to match the design aesthetic: generous whitespace, subtle borders, consistent hover states, clean typography.

Tool: Claude Code CLI

Prompt to Claude:

```

Polish all component styling:

Review and update these components:

1. Cards (all variants)
   - Border: 1px solid #E5E5E5
   - Padding: 24px
   - Border radius: 8px
   - Hover: subtle shadow (optional)
2. Buttons
   - Primary: solid background, white text
   - Secondary: outline, no fill
   - Ghost: no border, subtle hover bg
   - Padding: 12px 24px
   - Border radius: 6px
   - Hover: 150ms transition
3. Input fields
   - Border: 1px solid #E5E5E5
   - Focus: blue accent border (#0066CC)
   - Padding: 12px
   - Border radius: 6px
4. Typography
   - Headings: semibold, tight line-height
   - Body: normal weight, relaxed line-height
   - Muted text: #6B7280
5. Badges
   - Subtle background colors
   - Small padding: 4px 8px
   - Border radius: 4px
6. Spacing
   - Between sections: 48px (mb-12)
   - Between cards: 24px (mb-6)
   - Inside cards: 16-24px

Ensure consistency across all pages.

Use Shadcn components as base but customize to match design.
```

```

**Expected outcome:** Consistent, polished UI across all pages

**Verification:** Visual review of all pages for consistency

---

- [ ] **12.7 Add empty states**

**What we're doing:** Creating empty state components for pages with no data (e.g., no memory entries yet, no topics created). Empty states should be friendly and guide users on what to do next.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create empty state components:

File: src/components/ui/EmptyState.tsx

Create a reusable component that displays:

- Icon (large, subtle)

- Heading (what's missing)

- Description (why it's empty)

- Action button (what to do next)

Create specific empty states:

1. No memory entries: "Start learning to build your memory"

2. No topics: "Create your first topic to begin"

3. No evidence: "Complete an application to add evidence"

4. No GitHub repos selected: "Select repositories to track"

5. No weekly review yet: "Complete a week to see your review"

Design:

- Centered layout

- Large icon (96px)

- Muted colors

- Clear CTA button

Use across all relevant pages.

```

**Expected outcome:** Empty states guide users when no data exists

**Verification:** Visit pages with no data and see friendly empty states

---
```

- [ ] **12.8 Commit UI Polish**

What we're doing: Committing all UI polish, animations, and responsive design work.

Tool: Claude Code CLI

Prompt to Claude:

```
Create a git commit for UI polish:

git add .

git commit -m "$(cat <<'EOF'

feat: UI polish, animations, and responsive design

- Create design system tokens (colors, typography, spacing, animations)

- Integrate Framer Motion for smooth transitions (150ms, respects reduced motion)

- Implement responsive design for mobile, tablet, and desktop

- Add Focus Mode for distraction-free learning

- Create skeleton loading states for all data-fetching pages

- Polish all component styling (cards, buttons, inputs, badges)

- Add friendly empty states with clear CTAs

- Ensure consistent spacing and visual hierarchy

UI polish complete. App now has a cohesive, minimalist design

with smooth animations and works beautifully on all devices.

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -13
```

Expected outcome: Commit created for UI polish

Example output:

```
[feature/ai-learning-lab jkl3456] feat: UI polish, animations, and responsive design

 15 files changed, 892 insertions(+)

jkl3456 feat: UI polish, animations, and responsive design

ghi2345 feat: implement Settings and User Preferences

def1234 feat: implement Weekly Review and Adaptation system
```

...

```

**Verification:** `git log` shows new commit

---

## 13.0 Testing & Quality Assurance

**What we're doing:** Testing the entire application end-to-end. This includes: manual testing of all user flows, error handling verification, Groq API integration testing, database constraint testing, and creating a test user walkthrough to ensure the complete onboarding-to-learning flow works.

- [ ] **13.1 Test onboarding flow end-to-end**

**What we're doing:** Manually testing the complete onboarding flow from landing page to first Daily Learning Unit. Verifying all data is saved correctly and the flow is smooth.

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create a test checklist for onboarding flow:

Manual test steps:

1. Landing page

   - [ ] Landing page loads

   - [ ] Login buttons work (Google + GitHub)

2. Authentication

   - [ ] OAuth flow completes

   - [ ] User record created in database

   - [ ] Redirect to topic selection

3. Topic selection

   - [ ] Input field accepts topic name

   - [ ] Submit creates Topic record

   - [ ] Redirect to questionnaire

4. Questionnaire (8 sections)

   - [ ] All 8 sections display correctly

   - [ ] Progress indicator updates

   - [ ] Answers saved on each step

   - [ ] Multi-select questions work

   - [ ] Submit creates UserProfile record

   - [ ] Redirect to summary

5. Learning contract summary

- [ ] Summary generated by Groq API

- [ ] Summary reflects user's answers

- [ ] Confirm button works

- [ ] LearningStrategy created

- [ ] Redirect to graph review

6. Topic graph review

- [ ] Graph generated by Groq API

- [ ] Concepts grouped by difficulty

- [ ] Accordion expands/collapses

- [ ] Regenerate with feedback works

- [ ] Lock button creates DailyPlan

- [ ] Redirect to /dashboard/today

7. First Daily Learning Unit

- [ ] DLU content generated

- [ ] All sections render (concept, example, reflection, application)

- [ ] Complete Day button works

- [ ] MemoryEntry created

- [ ] Evidence modal appears (if application completed)

Test with at least 3 different topics (Docker, Go, React) to verify consistency.

Document any errors or issues in a file: TESTING_NOTES.md

```

****Expected outcome:**** Complete onboarding flow tested and verified

****Verification:**** All checklist items pass without errors

---

- [ ] ****13.2 Test Daily Learning Unit flow****

****What we're doing:**** Testing the complete daily learning flow including navigation, reflection, application, evidence capture, and day completion.

****Tool:**** Claude Code CLI

****Prompt to Claude:****

```

Create test checklist for DLU flow:

Manual test steps:

1. Today page load

- [ ] Fetches today's DLU correctly
- [ ] Shows correct day number (e.g., "Day 3 of 25")
- [ ] Concept name displayed
- [ ] All sections render

2. Content display
- [ ] Concept explanation renders (markdown)
- [ ] TL;DR shows if user prefers it
- [ ] Code examples have syntax highlighting
- [ ] Step-by-step instructions visible

3. Reflection section
- [ ] Prompts display correctly
- [ ] Textareas accept input
- [ ] Input saved to localStorage (persists on refresh)

4. Application section
- [ ] Starts collapsed
- [ ] Expands on click
- [ ] Task, guidance, expected output visible
- [ ] Checkbox tracks completion

5. Complete Day
- [ ] Button gathers reflection + application data
- [ ] MemoryEntry created in database
- [ ] DailyPlan.completedDays updated
- [ ] Evidence modal shows (if application completed)
- [ ] Success state displays
- [ ] "Move to Next Day" redirects correctly

6. Day navigation
- [ ] Previous/Next buttons work
- [ ] Query param (?day=N) loads correct day
- [ ] Buttons disabled appropriately (Day 1, Last day)

7. Caching
- [ ] Revisiting same day loads cached content
- [ ] Content consistent across visits

Test with multiple consecutive days (complete 3-5 days in a row).

```
```

Expected outcome: Daily learning flow tested and verified

Verification: All checklist items pass

- [ ] **13.3 Test Memory & Evidence system**

What we're doing: Testing memory timeline, filtering, evidence capture, and GitHub integration.

Tool: Claude Code CLI

Prompt to Claude:

```
Create test checklist for Memory & Evidence:

Manual test steps:

1. Memory Timeline
   - [ ] /dashboard/memory loads all entries
   - [ ] Entries sorted by date (newest first)
   - [ ] Cards display correctly (concept, date, reflection snippet)
   - [ ] Badges show (Applied, Evidence count)
   - [ ] Expand/collapse works
2. Filters
   - [ ] "All" shows all entries
   - [ ] "Applied" shows only entries with actionTaken
   - [ ] "Proof-backed" shows only entries with evidence
   - [ ] "By Topic" groups entries correctly
   - [ ] Counts in tabs are accurate
3. Evidence capture (manual)
   - [ ] Modal opens after application completion
   - [ ] "Paste output or link" works
   - [ ] Type dropdown functional
   - [ ] Label input optional
   - [ ] Submit creates EvidenceItem
   - [ ] Evidence appears in memory card
   - [ ] Skip closes modal without error
4. GitHub integration
   - [ ] Connect GitHub flow works
   - [ ] Repository selection saves
```

- [ ] Import modal shows commits/PRs

- [ ] Selecting items creates evidence

- [ ] Evidence marked as "verifiable"

- [ ] GitHub links clickable and correct

5. Edge cases

- [ ] No memory entries: empty state shows

- [ ] No evidence: empty state in proof-backed filter

- [ ] Multiple evidence items: all display correctly

Test with multiple memory entries and various evidence types.

```

**\*\*Expected outcome:\*\*** Memory & Evidence system tested

**\*\*Verification:\*\*** All checklist items pass

---

- [ ] **\*\*13.4 Test Weekly Review & Settings\*\***

**\*\*What we're doing:\*\*** Testing weekly review generation, adjustment suggestions, and settings updates.

**\*\*Tool:\*\*** Claude Code CLI

**\*\*Prompt to Claude:\*\***

```

Create test checklist for Review & Settings:

Manual test steps:

1. Weekly Review

- [ ] /dashboard/review generates review

- [ ] Progress summary accurate

- [ ] Insights cards display

- [ ] Completed vs skipped visualization correct

- [ ] Concepts covered list accurate

- [ ] Adjustment suggestion shows (if applicable)

- [ ] Next week preview displays

2. Adjustment suggestions

- [ ] "Apply Adjustment" updates LearningStrategy

- [ ] "Keep Current Pace" dismisses card

- [ ] Toast confirmation shows

- [ ] Changes take effect for future DLUs

3. Settings page

- [ ] All sections load

- [ ] Current values pre-filled

4. Learning Preferences

- [ ] Sliders update values

- [ ] Save button updates database

- [ ] Toast confirmation shows

5. Accessibility

- [ ] Checkboxes toggle correctly

- [ ] Radio buttons work

- [ ] Save persists changes

6. GitHub connection

- [ ] Status shows correctly (connected/not)

- [ ] Connect/disconnect works

- [ ] Repository selection link works

7. Preference persistence

- [ ] Changes persist after logout/login

- [ ] Future DLUs respect new preferences

- [ ] TL;DR appears if toggled on

Test updating multiple preferences and verifying they take effect.

```

**Expected outcome:** Review & Settings tested

**Verification:** All checklist items pass

---

- [ ] **13.5 Test error handling and edge cases**

**What we're doing:** Testing error scenarios to ensure the app handles failures gracefully (Groq API errors, database failures, invalid input, auth failures).

**Tool:** Claude Code CLI

**Prompt to Claude:**

```

Create test checklist for error handling:

Manual test steps:

1. Authentication errors

- [ ] Invalid session: redirects to login

- [ ] Expired token: prompts re-authentication
- [ ] Unauthorized access: shows 403 error

2. Groq API errors
- [ ] Rate limit: shows retry message
- [ ] Invalid response: shows error toast
- [ ] Timeout: shows timeout error
- [ ] Network error: shows connection error

3. Database errors
- [ ] Duplicate entries: handled gracefully
- [ ] Missing relations: caught and logged
- [ ] Invalid IDs: return 404

4. Form validation
- [ ] Empty required fields: validation error
- [ ] Invalid input: Zod error message
- [ ] Out-of-range values: range error

5. Edge cases
- [ ] No topics created: empty state
- [ ] All days completed: completion message
- [ ] No GitHub connection: can't import
- [ ] Invalid GitHub token: refresh prompt
- [ ] Very long reflection (>5000 chars): truncates or warns

6. Browser compatibility
- [ ] Works in Chrome
- [ ] Works in Safari
- [ ] Works in Firefox
- [ ] Works in Edge

Intentionally trigger errors (e.g., disconnect internet, invalid API key) and verify graceful handling.

```

**\*\*Expected outcome:\*\*** Error handling tested

**\*\*Verification:\*\*** All errors handled gracefully

---

- [ ] **\*\*13.6 Create final test report and commit\*\***

**\*\*What we're doing:\*\*** Documenting all test results, creating a test report, and committing the testing work.
```

Tool: Claude Code CLI

Prompt to Claude:

```

Create test report and commit:

Create file: TESTING_REPORT.md

Document:

1. Test Summary

   - Date tested: [today's date]

   - Tested by: [your name]

   - Environment: Development (http://localhost:3000)

2. Test Coverage

   - Onboarding flow: [✅ or ❌]

   - Daily Learning Unit: [✅ or ❌]

   - Memory & Evidence: [✅ or ❌]

   - Weekly Review: [✅ or ❌]

   - Settings: [✅ or ❌]

   - Error Handling: [✅ or ❌]

3. Issues Found

   - List any bugs or issues discovered

   - Severity: Critical / High / Medium / Low

   - Status: Fixed / Open

4. Browser Compatibility

   - Chrome: [✅ or ❌]

   - Safari: [✅ or ❌]

   - Firefox: [✅ or ❌]

   - Mobile (iOS): [✅ or ❌]

   - Mobile (Android): [✅ or ❌]

5. Performance Notes

   - Page load times

   - Groq API response times

   - Database query performance

6. Recommendations

   - Any improvements or fixes needed before production

After creating report, commit testing work:
```

git add .

git commit -m "$(cat <<'EOF'

test: complete end-to-end testing and QA

- Test onboarding flow (all 8 sections + topic graph + DLU)

- Test daily learning unit flow (navigation, reflection, application, completion)

- Test memory timeline and evidence capture (manual + GitHub)

- Test weekly review generation and adjustment suggestions

- Test settings updates and preference persistence

- Test error handling for all failure scenarios

- Verify browser compatibility (Chrome, Safari, Firefox)

- Document test results in TESTING_REPORT.md

All critical user flows tested and verified.

App ready for production deployment.

🤖 Generated with Claude Code

Co-Authored-By: Claude Sonnet 4.5 noreply@anthropic.com

EOF

)"

git log --oneline -14

```

**Expected outcome:** Test report created and testing committed

**Example output:**

```

[feature/ai-learning-lab mno4567] test: complete end-to-end testing and QA

 2 files changed, 143 insertions(+)

mno4567 test: complete end-to-end testing and QA

jkl3456 feat: UI polish, animations, and responsive design

ghi2345 feat: implement Settings and User Preferences

...

```

**Verification:** `git log` shows new commit, test report exists

---

## Relevant Files

**Configuration & Setup:**
```

- `package.json` - Project dependencies and npm scripts
- `next.config.mjs` - Next.js configuration
- `tailwind.config.ts` - Tailwind CSS theme and design tokens
- `tsconfig.json` - TypeScript compiler options
- `prisma/schema.prisma` - Complete database schema with 11 models
- `docker-compose.yml` - Local PostgreSQL container
- `.env` - Environment variables (not committed)
- `.env.example` - Environment variable template

****Authentication:****

- `src/lib/auth/auth.config.ts` - NextAuth.js configuration with OAuth providers
- `src/lib/auth/auth.ts` - NextAuth instance with Prisma adapter
- `src/lib/auth/utils.ts` - Auth helper functions (getSession, getCurrentUser)
- `src/middleware.ts` - Route protection middleware
- `src/app/api/auth/[...nextauth]/route.ts` - NextAuth API handlers

****Database:****

- `src/lib/db/prisma.ts` - Prisma client singleton
- `prisma/migrations/` - Database migration history

****Pages & Layouts:****

- `src/app/page.tsx` - Landing page
- `src/app/login/page.tsx` - Login page with OAuth buttons
- `src/app/(dashboard)/layout.tsx` - Dashboard layout with navigation
- `src/app/(dashboard)/today/page.tsx` - Daily learning page
- `src/app/(dashboard)/memory/page.tsx` - Memory timeline page
- `src/app/(dashboard)/settings/page.tsx` - Settings page
- `src/app/onboarding/page.tsx` - Topic selection
- `src/app/onboarding/questionnaire/page.tsx` - Multi-step questionnaire
- `src/app/onboarding/summary/page.tsx` - Learning contract summary
- `src/app/onboarding/graph/page.tsx` - Topic graph review

****Components:****

- `src/components/ui/*` - Shadcn/ui base components
- `src/components/onboarding/TopicInput.tsx` - Topic selection component
- `src/components/onboarding/QuestionnaireForm.tsx` - Multi-step form
- `src/components/onboarding/ProgressIndicator.tsx` - Progress bar
- `src/components/learning/DailyLearningUnit.tsx` - DLU display component
- `src/components/learning/ConceptCard.tsx` - Concept card component
- `src/components/learning/ReflectionPrompts.tsx` - Reflection UI
- `src/components/memory/MemoryTimeline.tsx` - Memory entries list
- `src/components/memory/MemoryEntryCard.tsx` - Individual memory entry
- `src/components/memory/EvidenceModal.tsx` - Evidence capture modal
- `src/components/memory/FilterTabs.tsx` - Timeline filters
- `src/components/shared/MarkdownRenderer.tsx` - Markdown with syntax highlighting

****API Routes:****

- `src/app/api/onboarding/profile/route.ts` - Save questionnaire answers
- `src/app/api/onboarding/summary/route.ts` - Generate learning contract

- `src/app/api/topics/route.ts` - Create topic
- `src/app/api/topics/[id]/graph/route.ts` - Generate topic graph
- `src/app/api/topics/[id]/graph/lock/route.ts` - Lock graph version
- `src/app/api/topics/[id]/plan/route.ts` - Get daily plan
- `src/app/api/learning/today/route.ts` - Get today's DLU
- `src/app/api/learning/[day]/complete/route.ts` - Mark day complete
- `src/app/api/memory/route.ts` - Get/create memory entries
- `src/app/api/memory/[id]/evidence/route.ts` - Add evidence
- `src/app/api/github/connect/route.ts` - GitHub OAuth connection
- `src/app/api/github/repos/route.ts` - List user repos
- `src/app/api/github/import/route.ts` - Import commits/PRs
- `src/app/api/weekly-review/route.ts` - Generate weekly review

****Groq Integration:****

- `src/lib/groq/client.ts` - Groq SDK client initialization
- `src/lib/groq/prompts.ts` - All Groq prompt templates
- `src/lib/groq/error-handler.ts` - Retry logic and error handling

****Utilities:****

- `src/lib/utils/validation.ts` - Zod schemas for validation
- `src/lib/utils/date.ts` - Date formatting helpers
- `src/types/index.ts` - Shared TypeScript types

Notes

First Time Using Claude Code?

1. Install Claude Code CLI: `npm install -g @anthropic-ai/claude-code`
2. Navigate to your project directory: `cd /path/to/ai-learning-lab`
3. Start Claude Code: `claude-code`
4. Copy-paste the prompts from each sub-task exactly as written
5. Check off tasks as you complete them: `- [ ]` → `- [x]`

Important Reminders

- ****Docker Desktop Required:**** Make sure Docker is installed and running before task 1.6
- ****OAuth Credentials:**** You'll need to create Google and GitHub OAuth apps (task 3.3)
- ****Groq API Key:**** Sign up at https://console.groq.com to get your API key
- ****Environment Variables:**** Never commit `.env` file - use `.env.example` as template
- ****Database Migrations:**** Always run `npx prisma migrate dev` after schema changes
- ****Git Commits:**** Commit after completing each parent task (prompts provided)

Testing

- Run dev server: `npm run dev` (starts on http://localhost:3000)
- Check database: `npx prisma studio` (opens GUI at http://localhost:5555)
- Run linter: `npm run lint`
- Format code: `npm run format`

- View Docker logs: `docker-compose logs -f postgres`

File Organization

- All source code in `src/` directory
- Pages in `src/app/` (Next.js App Router)
- Reusable components in `src/components/`
- Business logic in `src/lib/`
- Type definitions in `src/types/`

Progress Tracking

- Update this file after EACH sub-task completion
- Use git commits to create checkpoints (one per parent task)
- Test each feature before moving to the next parent task
- If stuck, refer to the PRD (`ai-learning-lab-prd-cc.md`) for detailed specifications

Common Commands Reference

```
# Development
npm run dev                          # Start dev server
npm run build                        # Production build
npm run lint                         # Run ESLint
npm run format                       # Format with Prettier

# Database
npx prisma studio                    # Open database GUI
npx prisma generate                  # Generate Prisma Client
npx prisma migrate dev               # Create and apply migration
npx prisma db push                   # Sync schema (dev only)

# Docker
docker-compose up -d                 # Start PostgreSQL
docker-compose down                  # Stop PostgreSQL
docker-compose logs -f postgres      # View logs
docker ps                            # List running containers
```

```
# Git

git status                          # Check status

git add .                           # Stage all changes

git commit -m "message"             # Commit with message

git log --oneline                   # View commit history
```

****END OF TASK LIST****

****Total Tasks:**** 14 parent tasks (0.0-13.0) with 118 detailed sub-tasks

****Estimated Implementation Time:**** This is a comprehensive project. Working with Claude Code full-time:

- ****Weeks 1-2:**** Foundation, Database, Auth, Onboarding (Tasks 0.0-5.0)
- ****Weeks 3-4:**** Topic Graph, Daily Learning Unit (Tasks 6.0-7.0)
- ****Weeks 5-6:**** Memory, Evidence, GitHub Integration (Tasks 8.0-9.0)
- ****Week 7:**** Weekly Review, Settings (Tasks 10.0-11.0)
- ****Week 8:**** UI Polish, Testing, QA (Tasks 12.0-13.0)

****Next Steps:****

1. Start with task 0.0 (Create feature branch)
2. Work through tasks sequentially - don't skip ahead
3. Check off each sub-task as you complete it
4. Commit after each parent task using the provided commit prompts
5. Test thoroughly before moving forward
6. Refer to the PRD for detailed specifications when needed

****Tips for Success:****

- Copy prompts exactly as written into Claude Code
- Test after each sub-task to catch issues early
- Use Prisma Studio to verify database changes
- Keep dev server running to see live updates
- Use the PRD as your source of truth for requirements

Good luck building AI Learning Lab! 🚀